WNDRVR
SUPPORT NETWORK

VXWORKS 7 SOCKETS PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Sockets Programmer's Guide, SR0650

17 June 2020

1. OVERVIEW

1. Introduction

VxWorks® 7 Sockets Programmer's Guide describes the use of VxWorks sockets as communication endpoints to which you can bind an address.

You use this address to access the socket. Each endpoint, or socket, exists within a space called a communications domain. Valid addresses for a socket are defined in terms of the socket's communication domain.

Source Code Files

The source code files for socket components are in the following directory:

installDir **/vxworks-7/pkgs_v2/os/service/socket-***x.x.x.x***/**

Next topic: Supported RFCs on page 1

2. Supported RFCs

Wind River sockets comply with relevant IETF RFCs.

- RFC 2553: *Basic Socket Interface Extensions for IPv6*
- RFC 3493: *Basic Socket Interface Extensions for IPv6*
- RFC 3542: *Advanced Sockets Application Program Interface (API) for IPv6*

Previous topic: Introduction on page 1
Next topic: Further Reading on page 1

3. Further Reading

Refer to additional documents for more information on sockets.

- VxWorks 7 Socket CAN Programmer's Guide
- IEEE Std. 1003.1 for information on the basic standard sockets API (documented as part of POSIX)
- *UNIX Network Programming, Volume 2, Second Edition* by W. Richard Stevens

Previous topic: Supported RFCs on page 1
Next topic: Sockets—the Wind River Implementation on page 1

4. Sockets—the Wind River Implementation

The Wind River Network Stack includes a standard socket interface to TCP and UDP.

Using sockets, you can do the following:

- Send and receive data over an IP network.
- Communicate with other processes.
- Access IP multicasting functionality.
- Review and modify the routing tables.

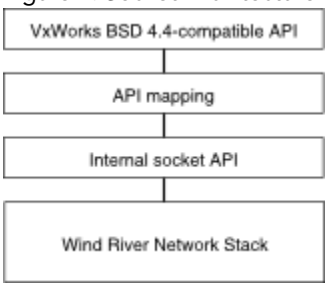
A socket is often implemented as a descriptor to which you bind an address. Each descriptor is associated with functions that can read or write information to or from the descriptor. The type of address that you can assign to the descriptor depends on the communications domain for the socket. For more information on socket types, see *Socket Types on page 3*, below.

Socket Architecture

The Wind River Network Stack uses a standard BSD-4.4 compatible socket interface to communicate with applications. The network stack in turn uses an internal API to communicate with this interface, but the internal API is not available for public use and does not provide optimum results. You should use the standard socket interface to program your application. This interface is implemented by the **sockLib** library.

The diagram below illustrates this architecture.

Figure 1. Socket Architecture of Wind River Network Stack



Note: The internal socket API shown is for internal use only. Use the BSD-compatible API implemented by **sockLib**.

Communications Domains

VxWorks supports the following socket domains:

- the IPv4 Internet domain (**AF_INET**) See Introduction on page 16 Internet Domain Sockets on page 16.
- the IPv6 Internet domain (**AF_INET6**) See Internet Domain Sockets on page 16.
- the routing domain (**AF_ROUTE**) See Routing Sockets on page 31.
- the netlink domain (**AF_NETLINK**) See Netlink and Rtnetlink Sockets on page 47.
- the packet domain (**AF_PACKET**) See Packet Sockets on page 62.

Raw Sockets

Raw sockets bypass the transport layer, which allows applications that use raw sockets to implement their own transport-layer processing from outside of the stack.

Socket Types

A socket's type describes the communication style that the socket uses. VxWorks includes the following socket communication styles or types:

SOCK_DGRAM

SOCK_DGRAM is bidirectional, unreliable, not sequenced, possibly duplicated, and message oriented.

The **SOCK_DGRAM** type is a connectionless communication style based on an exchange of datagrams. This style of communication is analogous to an exchange of letters (datagrams) using a postal service. The same datagram socket may communicate with several peers. The application is responsible for acknowledging its receipt of a datagram, not the underlying protocol. Datagram sockets do not automatically retransmit unacknowledged datagrams. They also do not guarantee that they will deliver datagrams in order, or that they will deliver a particular datagram only once. Internet domain datagram sockets operate over UDP, a transport layer protocol of the IP stack.

SOCK_STREAM

SOCK_STREAM is bidirectional, reliable, sequenced, non-duplicated, and stream-oriented.

The **SOCK_STREAM** type is a connection-oriented communication style characterized by the exchange of a continuous stream of data. This style of communication is analogous to a phone conversation. A **SOCK_STREAM** socket establishes a connection with a single peer. Data are sent as a stream of bytes that are delivered in order to the peer. If any segment of the stream fails to be acknowledged by its destination, the protocol will retransmit that segment. If, in spite of retransmission attempts, the recipient fails to acknowledge the data, the connection terminates and notifies the sender. Internet domain stream sockets operates over TCP, a transport layer protocol of the IP stack.

SOCK_SEQPACKET

SOCK_SEQPACKET is bidirectional, reliable, sequenced, non-duplicated, and message-oriented.

The **SOCK_SEQPACKET** type describes a communication style that is both connection oriented and message oriented. Being message oriented means that recipients of data over **SOCK_SEQPACKET** sockets receive data as discrete messages instead of as a stream of data. The Wind River Network Stack allows this socket type for sockets in the **AF_LOCAL** domain only.

SOCK_RAW

SOCK_RAW is usually bidirectional, interface and protocol-dependent; it provides access to internal network protocols and interfaces.

The **SOCK_RAW** socket is a socket with an undefined communication style. Its communication characteristics (reliable or not, sequenced or not, and so on) are determined by the services provided by the domain and protocol that you specify when you create the socket. For example, an application might create a raw **AF_INET** or **AF_INET6** socket and implement SCTP over it.

In the Wind River Network Stack, the primary use of **SOCK_RAW** sockets is within the **AF_ROUTE** domain, where tasks can use sockets to monitor and make changes to the contents of the local route table.

SOCK_RDM

SOCK_RDM is bidirectional, reliable, not sequenced, possibly duplicated, and message-oriented.

APIs

The **sockLib** library provides BSD 4.4-compatible API functions, which you can use to open, close, read, and write to sockets.

The **selectLib** library provides a BSD 4.3-compatible "select" facility, which you can use to pend, or wait for, activity on sockets, as well as pipes and serial devices. The network stack provides a backend to **selectLib**, so you can use this facility and call select( ) on a socket.

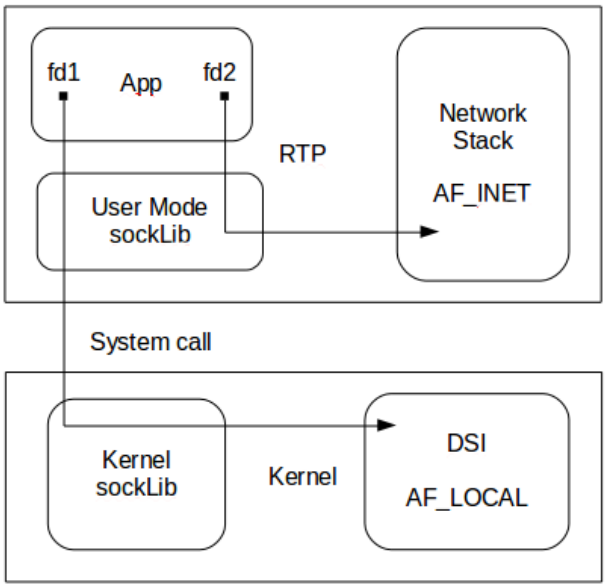
For information on socket API functions, see API Routines on page 13.

User Space Support

The VxWorks implementation of the BSD **sockLib** socket API is also available in user space. You can develop protocol stacks that execute in RTPs, and several copies of such stacks can run simultaneously in multiple RTPs. Examples of such protocol stacks include but are not limited to TCP/IP network stacks and Bluetooth stacks.

User mode **sockLib** allows an RTP application to create and use sockets from multiple protocol families where one family may be implemented in the kernel and another in user space. For example, in an application which acts as a proxy between Internet sockets and Local sockets, the proxy will open two sockets, one of type **AF_INET** and another of type **AF_LOCAL**. Assuming a network stack implements the **AF_INET** socket protocol in user space while the local socket protocol is implemented in kernel space, the figure below depicts this scenario:

Figure 2. An RTP application using user-space sockLib



User mode **sockLib** supports sockets both directly in user space, via a library call, and through a system call. The sequence when a socket is created is to:

1. Attempt to create the socket in user space. If creation fails because there is no backend added for the protocol family in question in the context of the RTP, then,
2. Attempt to create the socket in kernel space via a system call.
3. If socket creation in kernel space via a system call fails because there is no backend for the protocol family in question in the kernel, socket creation will fail.

Previous topic: Further Reading on page 1

2. CONFIGURATION AND BUILD WORKFLOW

1. Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VxWorks Source Build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VxWorks Image Project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source configuration options for sockets, see Source Build Configuration Options (VSB) Reference on page 5.

For a list of available kernel configuration components and parameters for sockets, see Kernel Configuration Components (VIP) Reference on page 6.

Next topic: Source Build Configuration Options (VSB) Reference on page 5

2. Source Build Configuration Options (VSB) Reference

The following table lists the socket VSB options.

Table 1. Socket VSB Options

Name	Description
SOCKET	The BSD socket library.
USER_MODE_SOCKET	Enables user mode socket support. This option requires that real-time process (RTP) support be enabled and provides **USER_MODE_IOS** support. It is disabled by default. For more information on user mode sockets, see Socket Configuration Parameters Reference on page 7. For more information on RTP, see the VxWorks 7 Configuration and Build Guide.
EPOLL	The VxWorks event polling facility (epoll), a scalable input and output event notification mechanism for monitoring network socket events. For more information, see Network Socket Event Polling on page 72.

Previous topic: Configuration and Build Workflow on page 5
Next topic: Kernel Configuration Components (VIP) Reference on page 6

3. Kernel Configuration Components (VIP) Reference

The following table lists the socket VIP components.

Table 1. Socket VIP Components

Name	Description
INCLUDE_IPNET_USE_NETLINKSOCK	Netlink is a network service that enables communication between the kernel and user-space processes. This component includes the routing socket interface and implements both NETLINK and RTNETLINK. See Netlink and Rtnetlink Sockets on page 47.
INCLUDE_IPNET_USE_ROUTESOCK	Routing sockets provide a two-way communication interface to the route table. See Routing Sockets on page 31.
INCLUDE_SOCKLIB	Socket Application Programming Interface (API). The **sockLib** library provides BSD 4.4-compatible API functions, which you can use to open, close, read, and write to sockets. If you do not want to implement your own domain handler, you do not need to include this component in your project. See Socket API Routines Reference on page 13.
INCLUDE_SC_SOCKLIB	This component includes sockScLib, a collection of socket API stubs (system call infrastructure) that allow RTPs to use socket calls even if the RTPs are in a different address space from the network stack. This component is included automatically when RTP support, **INCLUDE_RTP**, is included, and this component also requires the **INCLUDE_RTP** component.
INCLUDE_IPNET_USE_SOCK_COMPAT	Include this component in any build that uses the socket API. Along with the Socket support component, it provides the standard socket back end to the socket API. See Adding Standard Sockets Interface, and More on page 65.
INCLUDE_UN_COMP	A **SOCK_SEQPACKET**-based protocol for the **AF_LOCAL** domain. This component is not a network stack component. You can include COMP in a VxWorks image and use **AF_LOCAL** sockets even if you do not include a network stack. Before adding **INCLUDE_UN_COMP** to your VIP project, you must first include the **SERVICE_UN** component and rebuild the underlying VSB. See the VxWorks 7 Configuration and Build Guide for information on adding components to projects.
INCLUDE_IPNET_SOCKET	The standard socket implementation for the network stack. These sockets have the same semantics and use the same structures as the BSD 4.4 socket API.

Name	Description
	See Socket Configuration Parameters Reference on page 7 for a list of socket support parameters and their default values.

Previous topic: Source Build Configuration Options (VSB) Reference on page 5
Next topic: Optional Kernel Configuration Components Reference on page 7

4. Optional Kernel Configuration Components Reference

The following table lists optional components for your application.

Table 1. Optional Components

Name	Description
INCLUDE_IPSOCKPERF_CMD	Socket performance test tool shell commands. See Using the Socket Performance Shell Command on page 12.
INCLUDE_IPSOCKTEST_CMD	Socket test tool shell commands. See Getting Help for Socket Test Shell Commands on page 10.
INCLUDE_SELECT	The **selectLib** library provides a BSD 4.3-compatible select facility, which you can use to pend, or wait for, activity on sockets, pipes and serial devices. The network stack provides a backend to **selectLib**, so you can use this facility and call **select( )** on a socket.
INCLUDE_EPOLL	The VxWorks event polling facility (epoll), a scalable input and output event notification mechanism for monitoring network socket events. For more information, see Network Socket Event Polling on page 72.

Previous topic: Kernel Configuration Components (VIP) Reference on page 6
Next topic: Socket Configuration Parameters Reference on page 7

5. Socket Configuration Parameters Reference

The following table lists the socket configuration parameters.

Table 1. Socket Configuration Parameters

Parameter Name and Description	Default and Type
IPNET_SOCK_ADDR_NOTIFY	"1"

Parameter Name and Description	Default and Type
If you set this to "1", when an address is removed from a node on which socket applications are running, the stack sends an **ENETDOWN** failure message to all socket owners that bound their sockets to that address and that are blocked on an **accept( )** call. Set this to "0" to turn off this notification, in which case the socket owners will remain blocked on **accept( )** in such a circumstance.	char *
IPNET_SOCK_ANON_PORT_MAX The highest port number that the system may use as an ephemeral port number. The system assigns a port in the ephemeral port number range from **AnonPortMin** to **AnonPortMax** to a socket if that socket does not select its own port with a **bind( )** call.	"65535"" char *
IPNET_SOCK_ANON_PORT_MIN The lowest port number that the system may use as an ephemeral port number. The system assigns a port in the ephemeral port number range from **AnonPortMin** to **AnonPortMax** to a socket if that socket does not select its own port with a **bind( )** call.	"49152" char *
IPNET_SOCK_DEFAULT_RECV_BUFSIZE The default size of the socket receive buffer. Maximum size: INT_MAX (2147483647 bytes). The potential disadvantage of increasing the size of this buffer is that an unnecessary amount of memory may be assigned and never used. See the **setsockopt( )** reference entry for more	"60000" char *
IPNET_SOCK_DEFAULT_SEND_BUFSIZE The default size of the socket send buffer.	"100000" char *
IPNET_ROUTE_MAX_CLONE_COUNT Defines the maximum number of cloned route entries that might be present at any time. Most common usage of cloned routes are ARP and NDP entries.	"1000" char *
IPNET_ROUTE_NEIGH_STALE_CORRECT More correct but more expensive handling of STALE state. Ensures that even when sent to only by non-primary stack instances, a neighbor in STALE state globally transitions to DELAY state when any stack instance sends to the neighbor. This correct behavior can prevent brief interruption in traffic to the neighbor in some cases, but is not absolutely necessary and does impose additional PENTIUM4 load on the primary stack instance. Use "yes" or "no".	"yes" char *
IPNET_SOCK_ASSIGN_RANDOM Pseudo-random assignment of new sockets to stack instances. Use "yes" or "no". When this parameter is "no", new sockets are always assigned to the primary network stack instance (tNet0). When this parameter is "yes", each new sockets is assigned pseudo-randomly to one of the network stack instances.	"no" char *
IPNET_ROUTE_NEIGH_RATE_PERIOD	"100"

Parameter Name and Description	Default and Type
This parameter defines the rate limit period for the newly created neighbor entries in milliseconds.	char *
IPNET_ROUTE_NEIGH_RATE_LIMIT This parameter defines the limit of new neighbor entries \ that can be created in one neighbor rate limit period. The system cannot create new neighbors once this limit is reached. The default value is 0, which indicates that there is no maximum limit. A positive value is the limit number. Generally, this value is less than **IPNET_ROUTE_MAX_CLONE_COUNT**.	"0" char *

Previous topic: Optional Kernel Configuration Components Reference on page 7

3. SHELL COMMANDS

1. Introduction

You can use the socktest shell command to test socket functionality.

You can use the sockperf shell command to test socket performance

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Next topic: Getting Help for Socket Test Shell Commands on page 10

2. Getting Help for Socket Test Shell Commands

Command line help is available for socktest and its options.

About This Task

Procedure

1. List the available socket test options and their syntax, use the following command:

```
[vxWorks *]#
[vxWorks *]# socktest
NAME
      socktest - socket testing utility

SYNOPSIS
      socktest console
      [socktest] exit
      socktest open -d <domain> -t <type> -p <protocol>
      socktest close -s <socket> -f
      socktest bind -s <socket> [-a <address>] [-p <port>] [-i <interface name>]
      socktest setopt -s <socket> -l <level> -o <option>=<data>
      socktest getopt -s <socket> -l <level> -o <option> [-a <assoc_id>]
      socktest ioctl -s <socket> -o <ioctl>[=<data>] -i <interface name>
      socktest connect -s <socket> -a <peer address> -p <peer port> [-d <domain>]
      socktest listen -s <socket> -b <back log>
      socktest accept -s <socket>
      socktest getsockname -s <socket>
```


WNDRVR

```
      socktest getpeername -s <socket>
      socktest gethostbyname [-r] <name>
      socktest gethostbyaddr [-r] <address>
      socktest receive -s <socket> [-l <max length>] [-n <packets>] [-NPmorwc]
      socktest send -s <socket> -d <data> [-f <flags>] [-a <to address>] [-p <to port>] [-c <sm
>]
      socktest zerocopy_recv -s <socket> [-l <max length>] [-n <packets>]
                       [-r] [-a <from address>] [-p <from port>]
      socktest zerocopy_send -s <socket> -d <data> [-f <flags>] [-a <to address>] [-p <to port>
]
      socktest select -r <rsock> -w <wsock> -x <xsock>
      socktest shutdown [-r] [-w]
      socktest attach -s <socket> -d <domain> -t <type> -p <protocol>
      socktest if_nametoindex <ifname>
      socktest soevent_monitor -s <socket>
OPTIONS      -N <n> repeat the same command <n> times

[vxWorks *]#
```

2. List detailed help for the option you are interested in.

   For example, to get help for the attach option, enter:

```
# socketest attach
USAGE
     socktest attach -s <socket> -d <domain> -t <type> -p <protocol>

     OPTIONS
     -s specify socket number
     -d <domain>
               0  unspec
               2  inet
              17  route
              18  link
              19  packet
              28  inet6
               0  key

     -t <type>
               1  stream
               2  datagram
               2  dgram
               3  raw
              10  packet

     -p <protocol>
               1  icmp
               2  igmp
               4  ipv4
               6  tcp
              17  udp
             136  udplite
              41  ipv6
              50  esp
              51  ah
              58  icmpv6
             103  pim
```

```
            255  raw
              0  p_all
           2048  p_ip
           2054  p_arp
          33024  p_vlan
          34525  p_ipv6
          34915  p_ppoe_dis
          34916  p_ppoe_ses
```

Previous topic: Introduction on page 10
Next topic: Using the Socket Performance Shell Command on page 12

3. Using the Socket Performance Shell Command

The socket performance shell commands allow you to verify the performance aspects of your sockets.

About This Task

The command syntax is as follows:

```
sockperf [options] [address]
```

Procedure

To list the available command options, use the following command:

```
# sockperf
Network performance tool
usage:
   sockperf [options] [address]
      -6         use IPv6 (default is IPv4)
      -a         use passive open (default is active open)
      -e         echo test pattern on shell
      -L <sec>   linger time for all sockets (default is 3s)
      -l <bytes> number of bytes to write/read in each call (default 8192)
      -n <num>   number of buffers to write (default 2048)
      -P         create(transmit) or verify(receive) test pattern
      -p <port>  socket port (default 7373)
      -R         socket receive buffer size (default 16384)
      -r         receive data (not mutually exclusive with -t)
      -s <num>   number of sockets to use, must be same on both ends (default 1)
      -T         socket send buffer size (default 32767)
      -t         transmit data (not mutually exclusive with -r)
example:
   "sockperf -r -a" and "sockperf -t <addr>" should give about the same
   performance value as "ttcp -r" and "ttcp <addr>"
```

Previous topic: Getting Help for Socket Test Shell Commands on page 10

4. API ROUTINES

1. Socket API Routines Reference

The socket API (extensible interface) is the core component within this system.

It enables you to implement your own socket domain handler and mediates access to registered socket back ends. This is the interface you should use for all applications that require a socket domain handler and access to the socket back ends.

sockLib

The socket library (**sockLib**) implements the standard socket interface layer. This layer mediates access to registered socket back ends. If you include any of the other sockets components, this automatically registers their socket back ends with the standard socket interface layer. The **sockLib** API functions are defined in **/src/sockLib.c**. For example usage of the **sockLib** API functions, see **/user_src/sockLib.c.**

sockScLib

The socket system call library (**sockScLib**) implements a collection of socket API stubs that allow RTPs to use socket calls even if the RTPs are in a different address space from the network stack. The **sockScLib** system call infrastructure is defined in **/src/sockScLib.c**.

selectLib

The select library (**selectLib**) library provides a BSD 4.3-compatible select facility, which you can use to pend, or wait for, activity on sockets, as well as pipes and serial devices. The network stack provides a backend to **selectLib**, so you can use this facility and call select( ) on a socket. The **selectLib** API functions are defined in

installDir **/vxworks-7/pkgs_v2/os/core/io-***x.x.x.x* **/src/ios/selectLib.c**.

epollLib

For information on the epoll library (**epollLib**), see Event Polling API on page 73.

API Routines

The *sockLib API Routines* and *sockLib API Routines* tables list the API functions available to socket applications and gives a brief description of each function. For more information, see the reference entries for **sockLib** and **selectLib**.

Table 1. sockLib API Routines

Routine	Description
accept( )	Accept a connection from a socket.
bind( )	Bind a name to a socket.
connect( )	Initiate a connection to a socket.
connectWithTimeout( )	Attempt socket connection within a specified timeframe.
getpeername( )	Get the name of a connected peer.

Routine	Description
getsockname( )	Get a socket name.
getsockopt( )	Get socket options.
listen( )	Enable connections to a socket.
recv( )	Receive data from a socket.
recvfrom( )	Receive a message from a socket.
recvmsg( )	This function receives a message from a datagram socket. It may be used in place of **recvfrom( )** to decrease the overhead of breaking down the message-header structure (msghdr) for each message.
send( )	Send data to a socket.
sendmsg( )	Send a message to a socket. This function sends a message to a datagram socket. It may be used in place of **sendto( )** to decrease the overhead of reconstructing the message-header structure (msghdr) for each message. Cannot be called from an interrupt service routine (ISR) handler.
sendto( )	Send a message to a socket. Cannot be called from an interrupt service routine (ISR) handler.
setsockopt( )	Set socket options.
shutdown( )	Shut down a network connection.
socket( )	Open a socket.

Table 2. sockLib API Routines

Routine	Description
select( )	Pend on a set of file descriptors.
selectInit( )	Initialize the select facility.
selNodeAdd( )	Add a wake-up node to a **select( )** wake-up list.
selNodeDelete( )	Find and delete a node from a **select( )** wake-up list.
selWakeup( )	Wake up a task pended in **select( )**.
selWakeupAll( )	Wake up all tasks in a **select( )** wake-up list.

Routine	Description
selWakeupListInit( )	Initialize a **select( )** wake-up list.
selWakeupListLen( )	Get the number of nodes in a **select( )** wake-up list.
selWakeupListTerm( )	Terminate a **select( )** wake-up list.
selWakeupType( )	Get the type of a **select( )** wake-up node.

WNDRVR

5. INTERNET DOMAIN SOCKETS

1. Introduction

VxWorks uses Internet domain sockets for communication with remote nodes.

You can also use sockets in these domains to add or remove routing entries in the network stack's multicast forward information base (routing table).

In the Internet domains, the VxWorks socket support functions let you define socket endpoints using either IPv4 or IPv6 addresses.

- Sockets in the IPv4 domain, **AF_INET**, bind to names defined in terms of an IPv4 address and a port number.
- Sockets in the IPv6 domain, **AF_INET6**, bind to names defined in terms of an IPv6 address and a port number, and in some cases, a scope identifier.

Applications typically use sockets in the Internet domains to exchange information with peers on remote nodes, but you can also use an Internet domain socket to communicate with a peer on the local host. However, in VxWorks, there are more efficient mechanisms for communication between local tasks, such as **AF_LOCAL** sockets and message queues.

As much as possible, the socket API is IPv6/IPv4 agnostic. Thus, the **sockLib** calls have been modified internally to respond appropriately to an IPv6 address, but they do this only if the socket( ) call that created the socket specified the IPv6 address family. In addition, to pass in an IPv6 address, you need to use a **sockaddr_in6** structure that you have cast to a **sockaddr** structure.

Despite this adaptation to IPv6 addresses, there are some socket-mediated services that will not work with IPv6 without modification. The IPv4 multicast application in particular must be modified to support IPv6 multicasting, since that part of the socket API differs substantially between the two protocols. The address format is different for IPv6, and the socket options used to configure multicast (**MCAST**_*xxx* at the **IPPROTO_IP** level) uses a different data format. For IPv6, the level parameter passed to setsockopt( ) is **IPPROTO_IPV6**.

Configuring VxWorks for Internet Domain Sockets

To add socket functionality to your project, see Source Build Configuration Options (VSB) Reference on page 5 and Kernel Configuration Components (VIP) Reference on page 6.

Next topic: About Internet Domain Sockets on page 16

2. About Internet Domain Sockets

You can set up an Internet socket, connect it to a peer, exchange data with that peer, and shut down the socket.

A socket is a communications endpoint to which you can bind a name. Under VxWorks, a socket is implemented and managed using a socket descriptor, to which you can bind a name from the desired domain. To create a socket descriptor, call socket( ). To configure this socket descriptor for your particular use of that socket, call setsockopt( ). To bind a name to a socket descriptor, call bind( ).

For more information, see the following sections:

- Creating a Socket Descriptor on page 17
- Socket Options on page 17
- Binding a Socket to a Local Address on page 20
- Internet Domain Socket Connections on page 23

WNDRVR

- Sending and Receiving Data Using Internet Domain Sockets on page 24
- Closing or Shutting Down an Internet Domain Socket Connection on page 26
- Working with Internet Addresses on page 27
- Adding MD5 Encryption to a TCP Socket on page 27
- Adding TCP-AO Encryption to a Socket on page 28

Previous topic: Introduction on page 16
Next topic: Creating a Socket Descriptor on page 17

3. Creating a Socket Descriptor

To send and receive data over a socket, you need a socket descriptor. To create a socket descriptor, call socket( ).

As input, a VxWorks socket( ) call expects you to specify three things:

- a communications domain for the socket
- a socket type
- a protocol type

For the communications domain, specify either **AF_INET** or **AF_INET6**, depending on whether you want a socket in the IPv4 or IPv6 domain. For the socket type, use **SOCK_STREAM** to indicate a stream type socket. To indicate a datagram type socket, use **SOCK_DGRAM**. In some cases you should also specify a protocol type, but this is not always necessary.

For instance, for a stream socket in an Internet domain, you could specify the protocol type of **IPPROTO_TCP** (TCP provides the necessary services for a stream socket in the Internet domains). For datagram sockets, you could specify a protocol type of **IPPROTO_UDP** (UDP is the IP protocol that provides the necessary services for a datagram socket in the Internet domain). However, in these cases you do not need to specify either protocol type explicitly. Instead, you can set the protocol type to 0 (zero), which tells socket( ) to choose whatever transport layer protocol is best suited to the requested socket type and communications domain.

For raw sockets in the Internet domains, a protocol of zero indicates the IP protocol, **IPPROTO_IP**. Such a socket gives you direct access to IP. If you want an Internet domain raw socket that uses a different protocol, use an appropriate **IPPROTO**_*name* value as defined in **in.h**.

For example, to verify that an IP address is not already in use, a DHCP server implementation might want to send out an ICMP echo request on the socket to test the address. To do this, it would first create a raw socket in the IPv4 Internet domain and specify a protocol of **IPPROTO_ICMP** (in the IPv6 Internet domain, it would specify a protocol of **IPPROTO_ICMPV6**).

Previous topic: About Internet Domain Sockets on page 16
Next topic: Socket Options on page 17

4. Socket Options

After creating a socket descriptor, you may want to specify the socket options that configure the socket for your particular needs.

You can set most options with a call to setsockopt( ), although there is one “option” you set with an ioctl( ) call.

Using an ioctl( ) Call to Make the Socket Non-Blocking

To make a socket non-blocking, call ioctl( ) as follows:

```
on = 1; if (ioctl (mySocketDescriptor, FIONBIO, (int) &on) == -1)
    { /* your response to the ioctl call failure */ }
```

For input operations such as recv( ), recvfrom( ), and read( ), if no data is available in the socket receive buffer, in the case of a blocking socket (default), the process is put to sleep until data arrives. For non-blocking sockets, the call will return immediately with the error set to EWOULDBLOCK.

For output operations such as send( ), sendto( ), and write( ), the kernel copies data from the application's buffer into the socket's send buffer. If there is no room in the socket's send buffer, in case of blocking sockets, the process is put to sleep until there is enough room. For non-blocking sockets, the call will return immediately with the error set to EWOULDBLOCK.

Calling setsockopt( ) to Set Socket Options

The synopsis for setsockopt( ) is defined as follows:

```
STATUS setsockopt ( int s, /* target socket */ int level, /* protocol
    level of option */ int optname, /* option name */ const void *optval, /* pointer to option val
ue
    */ socklen_t optlen /* option length */ )
```

This function sets the options associated with a socket and any underlying protocols that enable the services accessible through the socket. Some setsockopt( ) calls you must make before calling connect( ) or bind( ). This is because some socket options affect the outcome of the bind( ) or the way the connect operation is formed. For example, some socket options let you restrict the bind( ) to a particular range of port numbers.

Some of the defaults for socket options in the Wind River Network Stack are as follows:

Table 1. Default Values for Socket Options

Option	Default
SO_LINGER	Off.
SO_REUSEADDR	Disabled.
SO_KEEPALIVE	Disabled.
SO_DONTROUTE	Disabled.
SO_RCVLOWAT	1
SO_SNDLOWAT	1
SO_ACCEPTCONN	No default, returns 1 if **listen( )** has been called on the socket, 0 otherwise.
SO_BROADCAST	enabled (Note: this is the opposite of the default in Linux)
SO_USELOOPBACK	Enabled.
SO_SNDBUF	IPNET_SOCK_DEFAULT_SEND_BUFSIZE

Option	Default
SO_RCVBUF	IPNET_SOCK_DEFAULT_RECV_BUFSIZE
SO_RCVTIMEO	Infinite.
SO_ERROR	No default, return and clear the last socket error.
SO_TYPE	No default, returns the second argument passed to **socket( )**.
SO_BINDTODEVICE	Not bound to a device.
SO_OOBINLINE	Off.

Note: You do not have to call setsockopt( ). You can use sockets without setting any socket options if the default options associated with a stream, datagram, or routing socket are appropriate to your needs. However, it is a good idea to know the default socket option values for options such as **SO_LINGER** and **SO_REUSEADDR** and their consequences for the behavior of the socket.

The level parameter of a setsockopt( ) call lets you specify whether the option applies to the socket layer or to an underlying protocol. The socket-level options (level is **SOL_SOCKET**) are the most generic options and make sense in the context of many sorts of sockets. Names (optname values) for these socket options are defined in **socket.h**. In addition to these socket-level options, setsockopt( ) also lets you pass options through to the underlying protocol or protocols that enable and implement the services that you access through the socket.

For example, there are many options associated with IPv4. These options control and configure access to IPv4-supplied functionality such as IPv4-based multicasting. To set these, specify a level of **IPPROTO_IP**. Names (optname values) for these socket options are defined in **in.h**.

The setsockopt( ) reference entry describes the most commonly used of the IP socket options listed above. Paralleling the socket options for IPv4, the Wind River Network Stack also has IPv6 socket options. For IPv6, you would assign level a value of **IPPROTO_IPV6** and use the optname values for these socket options as defined in **in6.h**.

The **IPPROTO_TCP** protocol and TCP_NODELAY options are also frequently accessed. These are described in the setsockopt( ) reference entry. Other socket option names for this protocol are defined in **tcp.h**. Some protocols allow for socket access and some do not.

Sizing the Socket Receive Buffer for UDP Packets

When sizing the socket receive buffer for UDP packets, consider the number of packets expected and the data payload of each packet.

When a packet arrives at an interface and is passed up to the socket layer it will cause pkt->maxlen amount of memory in the socket receive buffer to be used, no matter how much data is within the packet.

VxWorks is designed this way to prevent DoS attacks in which many packets with 0 length data payloads are sent to flood an interface. Each packet will always take up pkt->maxlen space in the buffer even if the payload is 0. The default size for pkt->maxlen is 1600 bytes. This behavior applies only to UDP packets, not TCP.

Set an appropriate option for SO_RCVBUF so that the socket receive buffer can accommodate the expected number of UDP packets.

Letting the Network Stack Size the Socket Backlog

A networking application has a backlog, which is the number of sockets that may have finished the TCP three-way hand-shake and are awaiting to be returned from accept( ) on the listening socket.

The best way to size this backlog is to let the network stack select the size. Almost all applications should set the last argument of listen( ) to zero. This value allows the network stack to select a reasonably sized backlog.

Previous topic: Creating a Socket Descriptor on page 17
Next topic: Binding a Socket to a Local Address on page 20

5. Binding a Socket to a Local Address

To bind a socket descriptor to a local address (sometimes called a name), call bind( ).

As input, a VxWorks bind( ) call expects you to specify three things:

- the socket to which you would bind a name
- the name, in the form of a sockaddr-like structure
- the length of the name

As input, bind( ) expects the socket name to be supplied using a pointer to a sockaddr structure. The layout of the sockaddr structure is not convenient for setting up all the information needed to specify a socket name in the Internet communications domain. Instead, use either a sockaddr_in structure or a sockaddr_in6 structure that you cast to a sockaddr structure when you make the actual bind( ) call.

Use the sockaddr_in structure for sockets in the IPv4 Internet communications domain. The structure definition for the **Ip_sockaddr_in** address type is defined in

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/ipcom/include/ipcom_sock.h**.

Use the sockaddr_in6 structure for sockets in the IPv6 Internet communications domain. The structure definition for the **Ip_sockaddr_in6** address type is defined in

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/ipcom/include/ipcom_sock6.h**.

Note: Sockets in the routing domain never require an explicit bind( ) call.

If you do not call bind( ) for the socket descriptor that you create, the first time you call connect( ) or sendto( ) for the socket, the system automatically binds for you. In such a case, the system chooses the port number for your socket from among the unused private port numbers (also known as dynamic or ephemeral port numbers), in the range of **AnonPortMin** to **AnonPortMax**. The network stack chooses the local IP address appropriately to reach the specified destination. This association of a local IP address with the socket is temporary in the case of a sendto( ) (and may change from packet to packet), but the system maintains its port number assignment until you close the socket. This approach is acceptable if, as is common in client applications, you do not need to know the number of the port to which your socket binds.

Letting the System Select Your Port Number

A bind( ) call on an Internet domain socket passes a socket address containing both a local IP address and a local transport-layer port number. You may specify either or both of these as zero. If you specify a port number of zero, the system chooses an available (nonzero) local port number; you can obtain the port number later if necessary by using the getsockname( ) call. If you specify an IP

address of zero, the socket endpoint will accept connections or datagrams destined to any of the host's local IP addresses (with the chosen destination port number).

Binding to Well-Known and Registered Ports

In the Internet communications domains, port numbers 0 through 1023 are reserved for use by well-known services associated with TCP. These are services such as echo and time. Ports 1024 through 49151 are reserved for services registered with IANA. If you implement a server for a well-known or registered service, bind your socket to the port assigned to that service, and then monitor that socket for requests from clients. For a list of the port assignments for well-known and registered services, see:

http://www.iana.org/assignments/port-numbers

Setting Up a sockaddr_in Structure to Store an IPv4 Address

This list is also useful if you are implementing a client of a well-known service. It tells you the port on the remote site to which you send requests. If you need to request a port number assignment for a new service, you can apply to IANA. You need do that only if you are creating a service that you will publish to the entire Internet. For a service offered within a network throughout which you can control how port numbers are used, you can choose a private port number within the range 49152 through 65535.

```
struct sockaddr_in
    {
    u_char sin_len;          /* The length of the address. */
    u_char sin_family;       /* The address family, AF_INET. */
    u_short sin_port;        /* The port number. */
    struct in_addr sin_addr; /* The IP Address. */
    char sin_zero[8];        /* Optional mask for address. */
    };
struct in_addr
    {
    in_addr_t s_addr;
};
```

The *in_addr_t* is an unsigned 32-bit integer; its value is the IPv4 address stored in network byte order

You must assign a value to *sin_len* . If you do not do this before you call, for example, sendmsg( ), such a call will fail with an EINVAL (0x22) error.

Setting Up a sockaddr_in6 Structure to Store an IPv6 Address

Consider the following code fragment:

```
struct addrinfo hints;
struct addrinfo * res;
int r;
#define STRINGIFY(expr) #expr
bzero ((char*) &hints, sizeof (struct sockaddr_in6));
hints.ai_family = AF_INET6;
r = getaddrinfo (serverName, STRINGIFY(SERVER_PORT_NUM), &hints, &result);
if (r != 0)
    {
    fprintf (stderr, "%s\n", gai_strerror (r));
    return ERROR;
    }
```


WNDRVR

```
bcopy (result->ai_addr, &serverAddr, result->ai_addrlen);
freeaddrinfo (result);
```

The getaddrinfo( ) call creates a socket address structure from the *serverName* and the **SERVER_PORT_NUM**. You can specify the *serverName* either as a string representation of an IPv6 address or as a name. If you configured the stack to include DNS, getaddrinfo( ) uses DNS to get the name if the host table search fails.

Calling getaddrinfo( ) for IPv4 or IPv6

The following example code illustrates the use of getaddrinfo( ) in a TCP client to establish a connection to a server:

```
#include <vxWorks.h>
#include <string.h>
#include <errnoLib.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sockLib.h>
#include <ioLib.h>

int connectToServer
        (
        char * serverName, /* "fe80::1:0:34ea%fei0", "www.foo.com", etc. */
        char * serviceName /* "ftp", "http", "41834", etc. */
        )
        {
        struct addrinfo   hints;
        struct addrinfo * firstres;
        struct addrinfo * res;
        int               rc;
        int               sock;
        int               size;

        memset (&hints, 0, sizeof (hints));

        hints.ai_family = AF_UNSPEC;     /* allow either IPv4 or IPv6 */
        hints.ai_protocol = IPPROTO_TCP; /* let's restrict to TCP */

        rc = getaddrinfo (serverName, serviceName, &hints, &firstres);
        if (rc != 0)
                {
#ifdef DEBUG
                int err = errno;
                printf ("getaddrinfo() failed, returned: %d (%s), errno=0x%x\n",
                                rc, gai_strerror (rc), err);
#endif
                return ERROR;
                }

        /*
         * Simple strategy: Try the returned addresses until they run out,
         * or find one which works.  Warning: if there are several addresses
         * returned, and none of them works, pause here a while...
         */

        res = firstres; /* There's guaranteed to be at least one ... */
```


WNDRVR

```
        do
                {
                sock = socket (res->ai_family, res->ai_socktype,
                                        res->ai_protocol);

                if (sock < 0)
                        continue;
                                * Set desired socket options. Set socket buffer sizes,
                 * and treat (unlikely) setsockopt() failures as non-fatal.
                 */

                size = 65536;
                setsockopt (sock, SOL_SOCKET, SO_SNDBUF, (char *) &size,
                                        sizeof (size));
                setsockopt (sock, SOL_SOCKET, SO_RCVBUF, (char *) &size,
                                        sizeof (size));

                if (connect (sock, res->ai_addr, res->ai_addrlen) == 0)
                        break;

                close (sock); /* failed to connect, try with next address */
                sock = ERROR;

                } while ((res = res->ai_next) != NULL);

        freeaddrinfo (firstres); /* free all the addrinfo structures */

        return sock;
        }
```

The example above establishes a client TCP connection to a service specified as a server host name (which may be either a numerical IPv4 or IPv6 address, or a partially- or fully-qualified domain name), and a service name (which may be a decimal numeric port number, or one of a few named services (such as "ftp", "telnet", or "http").

The return value from this connectToServer( ) function is **ERROR** on failure, or a socket file descriptor if successful. You can use the returned socket descriptor to communicate with the server according to the specified application-layer protocol over TCP.

The function uses getaddrinfo( ) to do the name/service look-up and to retrieve parameters such as the socket address, socket address length, socket domain, socket type, and socket protocol that it uses when it calls socket( ), bind( ), and connect( ).

RFC 3493 (https://tools.ietf.org/html/rfc3493) contains useful information on how to make efficient use of getaddrinfo( ).

Previous topic: Socket Options on page 17
Next topic: Internet Domain Socket Connections on page 23

6. Internet Domain Socket Connections

You send and receive data through a socket in roughly the same manner whether the socket is a stream or datagram socket, but before you can exchange data over a stream socket, you must establish a connection.

The functions used to create this connection assume a client-server relationship.

A client wishing to establish a connection with a server calls connect( ) on a socket it has created to communicate with its server. Whether the connect( ) call blocks if a connection is not immediately available depends on how you have configured the socket (see Socket Options on page 17.

If the socket is non-blocking, connect( ) returns immediately. If connect( ) succeeds in establishing a connection, it returns **OK**. Otherwise, it returns **ERROR** and sets errno to **EINPROGRESS** or **EALREADY** which indicates that the system is still attempting to complete the connection. You may then repeat the connect( ) call until the system establishes a connection, at which time connect( ) will again return **ERROR**, but the errno value will be **EISCONN**.

On the server side, the server must have called listen( ) on the listening socket it has created to listen for incoming client connections. The server can then call accept( ) on the socket to wait for a new client connection.

When the accept( ) call returns, the return value is either **ERROR** or a new socket descriptor. The accept( ) call may return ERROR under normal operation if the peer resets the connection after it has completed but before it is accepted. In such a case, errno is set to **ECONNABORTED**. The server should treat this as a non-fatal error, and call accept( ) again to await further client connections. The server should treat **EINTR** and **EWOULDBLOCK** errno values the same way. Other errno values set by accept( ) indicate an error that is likely unrecoverable.

If accept( ) returns a socket descriptor rather than **ERROR**, the server can use this descriptor to exchange messages with the client that requested the connection. The server may choose to pass this socket descriptor to a new task while the server returns its attention to accepting connection requests on the original socket.

Servers typically call accept( ) from within a "forever" loop. If there is no connection request pending, the accept( ) blocks until a connection request arrives on the socket. If this behavior does not meet your needs, you can configure the socket to be non-blocking (see Socket Options on page 17) or you can use select( ). If no clients are requesting a connection over the socket, and the socket is non-blocking, the server's accept( ) call returns **ERROR** with errno set to **EWOULDBLOCK**.

Using a connect( ) Call with a Datagram Socket

Calling connect( ) for a datagram socket does not cause any actual communication with the specified destination. It does configure the socket to assume the specified destination for all messages read from and written to the socket. This makes it possible to call send( ) and recv( ) on the datagram socket instead of sendto( ) and recvfrom( ). Connected datagram sockets will only accept received datagrams whose source matches the specified peer address in the connect call.

Previous topic: Binding a Socket to a Local Address on page 20
Next topic: Sending and Receiving Data Using Internet Domain Sockets on page 24

7. Sending and Receiving Data Using Internet Domain Sockets

While the mechanics of sending and receiving data on a socket are simple, forming or interpreting the message can be more complicated and varies according to the application protocol assumed by the sender and recipient.

For the most part, the mechanics of forming protocol-appropriate messages is beyond the scope of this document. An exception is the discussion of routing-socket messages provided in Routing Sockets on page 31.

Sending and Receiving Data without Regard to the Control Data

If you have called connect( ) on a socket descriptor, or have received the socket descriptor from an accept( ) call, you can exchange messages over the socket using send( ) and recv( ) or the generic write( ) and read( ) functions. All four of these functions are typically used with stream sockets. If you called connect( ) on a datagram socket, you can also use these functions to communicate over that socket.

For a datagram socket for which you have not called connect( ), you need to explicitly identify the communications partner on the other side of the socket. In such a situation, you can use sendto( ) and recvfrom( ) to exchange messages over the socket.

Note: In asynchronous sends, the sending task does not wait for confirmation from the stack that a route to the send's destination could actually be found. To change this behavior, disable the **IPNET_SOCK_SENDMSG_ASYNCHRONOUS_IN_NONBLOCK_MODE** macro.

Accessing and Setting the Control Data for a Socket Message

The sendmsg( ) and recvmsg( ) functions let you work with the entirety of the socket message—both its message header and its data payload. You can access the message header by using a msghdr structure, which is defined in **socket.h** as:

```
/*
 * Message header for recvmsg and sendmsg calls.
 * Used value-result for recvmsg, value only for sendmsg.
*/
struct msghdr
    {
    void *       msg_name;        /* optional address */
    socklen_t    msg_namelen;     /* size of address */
    struct       iovec * msg_iov; /* scatter/gather array */
    int          msg_iovlen;      /* # elements in msg_iov */
    void *       msg_control;     /* ancillary data, see below */
    socklen_t    msg_controllen;  /* ancillary data buffer len */
    int          msg_flags;       /* flags on received message */
};
```

When you use the sendmsg( ) and recvmsg( ) functions with a datagram socket for which you have not called connect( ), use the msg_name field to specify the address of the communication partner on the other side of the socket.

The advantage of using sendmsg( ) and recvmsg( ) is that you can access the control (ancillary) information associated with the message, and not merely the message payload data. The msg_control field points to a buffer containing the ancillary data. You can access the data objects in this control buffer using cmsghdr structures, which **socket.h** defines as:

```
/*
 * Header for ancillary data objects in msg_control buffer.
 * Used for additional information with/about a datagram
 * not expressible by flags. The format is a sequence
 * of message elements headed by cmsghdr structures.
 */
struct cmsghdr
        {
        socklen_t  cmsg_len;    /* data byte count, including hdr  */
        int        cmsg_level;  * originating protocol            */
        int        cmsg_type;   /* protocol-specific type          */
                                                    /* followed by u_char cmsg_data[]; */
        };
```

To make it easier to work with these message headers, **socket.h** defines the macros CMSG_FIRSTHDR( ), CMSG_DATA( ), and CMSG_NXTHDR( ) as follows:

```
/* given pointer to struct cmsghdr, return pointer to data */
#define CMSG_DATA(cmsg)  ((u_char *)(cmsg) + \
         _ALIGN(sizeof(struct cmsghdr)))

/* given pointer to struct cmsghdr, return pointer to next cmsghdr */
#define CMSG_NXTHDR(mhdr, cmsg) \
```

```
 (((caddr_t)(cmsg) + _ALIGN((cmsg)->cmsg_len) + \
   _ALIGN(sizeof(struct cmsghdr)) > \
         (caddr_t)(mhdr)->msg_control + (mhdr)->msg_controllen) ? \
         (struct cmsghdr *)NULL : \
         (struct cmsghdr *)((caddr_t)(cmsg) + _ALIGN((cmsg)->cmsg_len)))

#define CMSG_FIRSTHDR(mhdr) ((struct cmsghdr *)(mhdr)->msg_control)
```

This level of access is convenient when implementing a service such as ping( ). If you are merely exchanging data and you are not interested in the control data, it may be better to use the send( )/sendto( )/write( ) and recv( )/recvfrom( )/read( ) functions.

Previous topic: Internet Domain Socket Connections on page 23
Next topic: Closing or Shutting Down an Internet Domain Socket Connection on page 26

8. Closing or Shutting Down an Internet Domain Socket Connection

When a client or server terminates conversation with the communication partner, it can break the connection by calling shutdown( ) or its generic near equivalent, close( ).

Of the two functions, shutdown( ) provides the greater flexibility. Using shutdown( ), you can break the send side of the socket connection, the receive side of the socket connection, or both. Wind River recommends, however, that if you call shutdown( ), you follow it with a call to close( ) to free memory resources associated with the socket.

The VxWorks kernel contains globally accessible file descriptors in the task-independent address space. For this reason, you must take precautions when closing a file descriptor (a socket descriptor is a variety of file descriptor). In the VxWorks kernel, one task must not close a file descriptor that another task is using, as this is not a safe operation.

When one task closes another, an operation in progress on the descriptor might return an error but also might cause an exception or other ill effects. Also, if an operation is performed on the descriptor after it is closed, the operation would normally return an error, but it could also affect another socket or file if the system reuses the descriptor after the descriptor is closed.

To wake up a task that is blocked on a socket, either use shutdown( ), set a timeout on the socket function on which the task is blocked, or deliver an event (for example, a message) over the socket on which the task is blocked. You can also set up a semaphore-based locking mechanism that prevents the close while an operation is pending on the descriptor.

If a network stack task (Task A) is blocked in a socket call and you want to end that task from another task (Task B), set a flag from task B that tells task A to delete itself and call

```
shutdown(sock, SHUT_RDWR)
```

from Task B, where *sock* is the socket that Task A sleeps on.

> **Note:** Do not close a socket descriptor in one task that is still in use by another task, because this practice is not safe in VxWorks. (The same is true of all file descriptors.) To wake up a task that is blocked waiting for an event on a socket, you may use shutdown( ) but not close( ), as shutdown( ) leaves the socket descriptor valid. You must then use some application-specific synchronization mechanism to ensure that all application tasks are done with it before the socket is closed.

Latency in Closing Sockets

If you enabled the SO_LINGER socket option and set the l_linger member of the struct linger argument, there can be a degree of latency associated with closing a socket connection. Calling close( ) initiates the tear-down of the session (in TCP this adds a FIN segment to the transmission queue), but when close( ) returns, the session is not yet closed and will not be in the **CLOSED** state until

tear-down negotiation is complete and the session is reset by one of the peers. The local pool will perform this reset if it no peer does it in the time specified by the SO_LINGER socket option.

By default, the SO_LINGER socket option is not enabled. To enable it, pass setsockopt( ) a struct linger argument with its l_onoff member set to 1, and its l_linger member set either to zero or to a timeout value specified in seconds. If l_linger is zero, the close( ) operation forces an abrupt close on the connection and sends a RST segment to the TCP peer. If l_linger specifies a nonzero timeout, the close( ) call will block until either the peer acknowledges all outstanding sent data and the local side's FIN, or until the specified l_linger timeout, or until TCP itself times out the connection (whichever occurs first). On the other hand, if you do not enable the SO_LINGER option for the socket, the close( ) call will return immediately, but TCP will initiate its graceful shutdown handshaking. If the local side did the close first, the connection will normally enter TIME_WAIT state on the local side and remain there for a timeout period of two times the maximum segment lifetime (MSL).

Previous topic: Sending and Receiving Data Using Internet Domain Sockets on page 24
Next topic: Working with Internet Addresses on page 27

9. Working with Internet Addresses

The Wind River Network Stack provides functions in **hostLib** and **inetLib** to convert host names to their corresponding numerical IP addresses and back.

getaddrinfo( )

Node name-to-address translation.

getnameinfo( )

Translate a socket address to a node name and service name.

gethostbyname( )

Name-to-address translation function.

gethostbyaddr( )

Address-to-name translation function.

Unless you are concerned with image size, the best functions to use are getaddrinfo( ) and getnameinfo( ), since they apply to both IPv4 and IPv6. Among the **hostLib** functions, hostGetByName( ) and hostGetByAddr( ) are IPv4-only except when the result of a query can be obtained locally—for example the required host IPv6 information that has been added to the host table by hostAdd( ).

Previous topic: Closing or Shutting Down an Internet Domain Socket Connection on page 26
Next topic: Adding MD5 Encryption to a TCP Socket on page 27

10. Adding MD5 Encryption to a TCP Socket

You can use the IP_TCP_X_MD5_SIGNATURE_KEY option to add MD5 encryption to a TCP socket. This option sets the MD5 key that will be used when creating MD5 signatures, as described in RFC 2583, Guidelines for Next Hop Client (NHC) Developers. This capability permits two peers that share a secret key or pass-phrase to authenticate each TCP segment on a session using a keyed MD5 hash.

This option is defined at the **IPPROTO_TCP** level. Setting a key with a length of 0 will disable this option. The argument is a shared secret (the local and remote peer must use the same secret) in the form of a C-string (char *), and the length passed to setsockopt( ) or getsockopt( ) is what strlen( ) would return if it were applied to the shared secret.

A common use case for this option is to encrypt communication between two border gateway patrol (BGP) routers. The routers set the same shared secret on their TCP sockets before calling connect( ) (on the client) and accept( ) (on the server). To set this secret, the routers call setsockopt( ) as follows:

```
setsockopt(sock, IPPROTO_TCP, TCP_X_MD5_SIGNATURE_KEY, ptr_to_shared_secret, length_of_shared_secret)
```

IP_TCP_X_MD5_SIGNATURE_KEY is defined in

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/ipcom/include/ipcom_sock.h**.

Previous topic: Working with Internet Addresses on page 27
Next topic: Adding TCP-AO Encryption to a Socket on page 28

11. Adding TCP-AO Encryption to a Socket

The TCP Authentication encryption option (TCP-AO) is defined by RFC 5925.

About This Task

Compared with MD5 it uses stronger Message Authentication Codes (MACs) and provides more information about the security associated with TCP connections. It also protects against replays for TCP connections, regardless of how long they have been in operation. The Wind River implementation of TCP-AO can be used with a static *Master Key Tuple* (MKT) configuration; it does not support an external, out-of-band MKT management mechanism.

Procedure

1. Create an a VSB project.

   Create an a VSB project and define **IPTCP_USE_AO_CMD** in

   installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/iptcp/config/iptcp_config.h**

2. Build the VSB project.
3. Create a VIP project.

   Create a VIP project based on the VSB and add the **INCLUDE_IPTCPAO_CMD** component.

4. Build the VIP project.

Next Steps

For managing TCP-AO at runtime (adding and deleting MKT keys, and so on), you can use the tcpao command from the shell, and iptcp_ao_api_mkt_add( ) and related functions programmatically. The API reference includes example code (to use the sample code you must define **IPTCP_AO_EXAMPLE**).

Previous topic: Adding MD5 Encryption to a TCP Socket on page 27

6. LOCAL DOMAIN SOCKETS

1. Introduction

Sockets in the local domain, **AF_LOCAL**, bind to names modeled after the names of files in a file system (no actual file system is necessary to use **AF_LOCAL** sockets).

In VxWorks, local domain socket names are implemented over Wind River's Connection-Oriented Message Passing (COMP) protocol. This protocol provides an optimized transport (better performance than **AF_INET** or **AF_INET6**) for applications that use the socket API to communicate between tasks that are running on the same node. This method is able to communicate between RTPs and the kernel, or among tasks running in different RTPs.

Under VxWorks, local domain sockets (**AF_LOCAL** sockets) are based on a protocol called COMP. This protocol enables the **SOCK_SEQPACKET** communication style, which is bidirectional, reliable, and message-based.

Configuring VxWorks for Local Domain Sockets

To add local domain socket functionality to your project, see Source Build Configuration Options (VSB) Reference on page 5 and Kernel Configuration Components (VIP) Reference on page 6.

The **INCLUDE_UN_COMP** component is a **SOCK_SEQPACKET**-based protocol for the **AF_LOCAL** domain. See the table *Socket VIP Components* in Kernel Configuration Components (VIP) Reference on page 6 for **INCLUDE_UN_COMP** details.

Next topic: Working with Local Domain Sockets on page 29

2. Working with Local Domain Sockets

To create a VxWorks image that includes local domain sockets, you must include the **INCLUDE_UN_COMP** component in your build.

This component pulls in the COMP implementation. See the table *Socket VIP Components* in Kernel Configuration Components (VIP) Reference on page 6 for **INCLUDE_UN_COMP** details.

Note that this component is not a network stack component and that COMP is independent of the network stack. You can include COMP in a VxWorks image and use **AF_LOCAL** sockets even if you do not include a network stack.

The Wind River Network Stack implements the standard socket API functions such as socket( ), bind( ), and listen( ). It also implements the standard file system API functions: read( ), write( ), close( ), ioctl( ), and select( ).

Because the **AF_LOCAL**/**COMP** sockets use a connection-oriented communication style, you must complete a bind-listen-connect-accept sequence before you can transmit data on the socket. To transmit and receive data over the socket, use the send( ) and recv( ) functions. The Wind River Network Stack implements the setsockopt( ), getsockopt( ), and ioctl( ) functions.

Setting up a Local Domain Socket

Under VxWorks, **COMP** manages the name space for an **AF_LOCAL** domain socket. Thus, when creating names for **AF_LOCAL** sockets, you must conform to the expectations of the **COMP** system, which is a name in the format:

/comp/socket/0xNumber

The /comp/socket/ part is an invariable prefix; the 0xNumber is a string representation of a 16-bit number in hexadecimal format. The Wind River Network Stack does not allow any other format for the name.

Use this name when you bind( ) a name to a socket and when you connect( ) to a listening socket. Supply this name by using a sockaddr_un structure:

```
#define UN_SOCKADDR_UN_PATH_SIZE 104

struct sockaddr_un                                  /* LOCAL (UNIX) family address */
        {                                               /* --------------------------- */
        uint8_t  sun_len;                               /* 0x00: structure size        */
        uint8_t  sun_family;                            /* 0x01: address family        */
        char     sun_path [UN_SOCKADDR_UN_PATH_SIZE];   /* 0x02: actual address        */
        };                                              /* 0x6A: TOTAL SIZE            */
```

This structure is defined in **un.h**. You will need to include **un.h** as well as **sockLib.h**, the include file for the standard socket API:

```
#include <sockLib.h> /* standard socket API */
#include <sys/un.h>  /* struct sockaddr_un definition */
```

The following example sets up a sockaddr_un structure for an **AF_LOCAL** socket:

```
mySockaddr_un.sun_len = sizeof (struct sockaddr_un); /* 106 bytes */
mySockaddr_un.sun_family = AF_LOCAL;
bcopy ("/comp/socket/0x1234", mySockaddr_un.sun_path, 20);
```

Previous topic: Introduction on page 29

7. ROUTING SOCKETS

1. Introduction

Routing sockets provide a two-way communication interface to the route table. Using a routing socket, an application can monitor and make changes to the contents of the routing table.

Traditionally, a route table stored only one entry per destination network. For IPv4, a destination network is identified by an IPv4 address and a netmask value. For IPv6, a destination network is identified by an IPv6 address and a prefix value. A route table entry associates this destination with a gateway on a local network or with an interface on a local network. In previous route table implementations, adding a route entry to the table failed if the table already contained an entry for that destination.

The route table in the Wind River Network Stack stores multiple same-destination routes that differ by gateway value or network mask. It is possible to add a route entry to a table that already contains an entry for that destination, provided the new entry differs from the existing entry (or entries) in its gateway value or network mask. Policy routing adds even more flexibility to a lookup by adding type of service (TOS) (IPv4), traffic class (IPv6), protocol, source address or network, destination address or network, and traffic flow (IPv6) as selectors.

When IP queries the route table, it expects to get back only one route. To satisfy this expectation, the route table ranks multiple same-destination routes. The highest ranked route, known as the primary or representative route for the destination, is the route reported to IP. How the system ranks same-destination routes depends on how you build the code.

Ranking Routes Using ECMP Routing

(rmx_hopcount). The ones with lower hopcount have higher precedence. If there are multiple same-destination entries with the same hopcount, selection is done through an algorithm called Equal-Cost Multipath (ECMP) which is described in RFC 2991, Multipath Issues in Unicast and Multicast Next-Hop Selection. Two of the algorithms described in RFC 2991 are implemented:

- Modulo-N Hash
- Highest Random Weight

Modulo-N Hash is the fastest, but adding or removing a same-destination route tends to change which route entry is returned for a specific traffic flow.

Highest Random Weight always returns the same route entry for a specific flow after a remove operation unless the entry that used to be returned was the one removed. The same entry or the new entry is selected after an add operation. The probability for the new entry to be selected over the old one is:

$1/N$

where N is number of same-destination entries, including the added entry.

Configuring VxWorks for Routing Sockets

To add the routing socket functionality to your project, see Source Build Configuration Options (VSB) Reference on page 5 and Kernel Configuration Components (VIP) Reference on page 6.

The **INCLUDE_IPNET_USE_ROUTESOCK** component includes the routing socket interface.

There are no configuration parameters or externally callable functions directly associated with this component

Next topic: Setting up a Routing Socket on page 32

2. Setting up a Routing Socket

Setting up a routing socket requires a socket( ) call. There is no need to make an explicit bind( ) or connect( ) call for a routing socket.

About This Task

After creating a routing socket, you may want to configure the socket options.

Procedure

1. Create a routing socket.

   To create a socket descriptor for use as a socket in the routing domain, call socket( ) with a domain value of **AF_ROUTE**, a type value of **SOCK_RAW** (raw protocol interface), and a protocol value of 0 (zero).

   For example:

   ```
   myRouteSocket = socket (AF_ROUTE, SOCK_RAW, 0);
   if (routeSocket < 0)
       {
       /* your response code for the socket call failure */
       }
   ```

   In most cases, you should supply a protocol argument of zero (0) for **AF_ROUTE** sockets. This is the most common value for the third argument of the socket( ) API. It instructs the network stack to use the default protocol for the selected domain and socket type.

2. Set options on the socket.

   After you have the socket descriptor, you can make it non-blocking by making an **FIONBIO** call to ioctl( ).

   For example:

   ```
   on = 1;
   if (ioctl (myRouteSocket, FIONBIO, (int) &on) == -1)
       {
       /* your response code for the ioctl call failure */
       }
   ```

   Another useful option is SO_USELOOPBACK, which is enabled by default. When this option is cleared, you do not receive the response to the messages that you write to your socket.

   ```
   on = 0;
       if (setsockopt (myRtSock, SOL_SOCKET, SO_USELOOPBACK, (char *)&on,
           sizeof (on)) == ERROR) {
           /* your response code for the setsockopt call failure */
       }
   ```

   A routing socket typically only sends a response if the requested operation fails. The only exceptions are the GET messages, which also send a response with the retrieved results (if successful). Disabling this option requires a separate routing socket to read the search results.

Previous topic: Introduction on page 31

Next topic: Working with Routing Socket Messages on page 33

3. Working with Routing Socket Messages

When preparing or processing a routing socket message, you can assume that it consists of a fixed-length header followed by up to eight socket address structures.

For most routing socket messages, the fixed-length header is described by an rt_msghdr structure, which is defined in

installDir **/vxworks-7/pkgs_v2/net/net_base/h/net/route.h**

This structure describes the fixed length message header for all but the following routing socket message types:

- **RTM_NEWADDR**
- **RTM_DELADDR**
- **RTM_IFINFO**
- **RTM_IFANNOUNCE**

For **RTM_NEWADDR** or **RTM_DELADDR** messages, the message header is described by the ifa_msghdr structure defined in **if.h**.

For **RTM_IFINFO** messages, the message header is described by the if_msghdr structure defined in **if.h**.

For **RTM_IFANNOUNCE** messages, the message header is described by the if_announcemsghdr structure defined in **if.h**.

Previous topic: Setting up a Routing Socket on page 32
Next topic: Case/Switch Processing for Received Messages on page 33

4. Case/Switch Processing for Received Messages

The first three members of the routing socket message header structures are all of the same type and carry the same meaning.

Thus, when a message first arrives on a socket, you can read the routing socket message type value using any structure overlay that is convenient. To make it easier to do this, you could receive the message into a union. Consider the following code fragment skeleton:[1] on page 34

```
#include <net/if.h>
#include <net/route.h>

/* sizeof the msglen, version and type fields that all AF_ROUTE messages must contain */ #define A
F_ROUTE_MIN_MSG_LEN 4

union af_route_msg {
   struct if_msghdr    if_hdr;
   struct ifa_msghdr  ifa_hdr;
   struct rt_msghdr   rt_hdr;
   struct if_announcemsghdr if_announcehdr; };

int input_af_route_msg(void *msg_ptr, int msg_len) {
   union af_route_msg *msg = msg_ptr;

   if (msg_len < AF_ROUTE_MIN_MSG_LEN)
```


WNDRVR

```
            /* Message is too short for being a AF_ROUTE message */
            return MALFORMED_MSG_ERROR_CODE;

  if (msg_len < msg->rt_hdr.rtm_msglen)
          /* Message is truncated or malformed */
          return TRUNCATED_MSG_ERROR_CODE;

   switch (msg->rt_hdr.rtm_type)
   {
   case RTM_ADD:
            /* Handle add message, reference message through msg->rt_hdr */
            break;
   case RTM_NEWADDR:
          /* Handle new address message, reference message through
          msg->ifa_hdr */
          break;
   case RTM_IFINFO:
          /* Handle interface status change, reference message through
          msg->if_hdr */
          break;
   case RTM_IFANNOUNCE
          /* Handle interface status change, reference message through
          msg->if_announcehdr */
          break;
        /* case statements for all other messages types goes here */
   default:
            /* Unsupported or invalid message type */
            return INVALID_MSG_TYPE_ERRO_CODE;
   }
}
```

Previous topic: Working with Routing Socket Messages on page 33
Next topic: Types of Routing Socket Messages on page 34

[1] on page 33

Although this union is useful for receiving messages, it would waste resources to use it when creating a routing socket message.

5. Types of Routing Socket Messages

Routing sockets on a VxWorks target running a host stack build use the standard set of routing socket messages described in the routing sockets section of TCP/IP Illustrated, Volume 2, with some exceptions.

For instance, interface indices are 32 bits, the rt_msghdr structure has a new element with a route table index associated with virtual router functionality, and the rt_metrics and if_data substructures occurring in rt_msghdr and if_msghdr messages are different. See the **route.h** and **if.h** header files for the structure definitions.

Constants for the message types are defined in **route.h**. The key at the top of the list indicates the message structure and whether the message can be sent or received by the network stack.

```
/* T = message can be sent to IPNET
   F = message can be sent from IPNET
   1 = message format is struct rt_msghdr
   2 = message format is struct ifa_msghdr
   3 = message format is struct if_msghdr
```

```
   4 = message format is struct if_announcemsghdr
 */
#define RTM_ADD        0x1   /* TF1  Add route */
#define RTM_DELETE     0x2   /* TF1  Delete route */
#define RTM_CHANGE     0x3   /* TF1  Change in metric or flags */
#define RTM_GET        0x4   /* TF1  Report metrics and other route information */
#define RTM_LOOSING    0x5   /*  F1  Kernel suspects route is failing */
#define RTM_REDIRECT   0x6   /*  F1  Kernel told to use a different route */
#define RTM_MISS       0x7   /*  F1  Lookup failed on this address */
#define RTM_LOCK       0x8   /* TF1  Lock specified metric */
#define RTM_OLDADD     0x9   /* Unsupported! */
#define RTM_OLDDEL     0xa   /* Unsupported! */
#define RTM_RESOLVE    0xb   /*  F1  Request to resolve destination to link-layer address */
#define RTM_NEWADDR    0xc   /* TF2  Address is added to interface */
#define RTM_DELADDR    0xd   /* TF2  Address is removed from interface */
#define RTM_IFINFO     0xe   /*  F3  Interface status or flag(s) is changing */
#define RTM_IFANNOUNCE 0x10  /*  F4  Interface attached/detached */
```

The network stack also includes two extended routing socket messages that you can use to create or destroy a virtual router. For more information, see the sections on the virtual router in the VxWorks 7 Network Stack Programmer's Guide. The extended messages are:

```
#define RTM_NEWVR 0xf1 /* TF1 Add a new virtual router */
#define RTM_DELVR 0xf2 /* TF1 Delete a virtual router, the default
                          (0)route table cannot be removed */
```

The following sections describe these two messages as well as the other message types.

RTM_ADD

You can both read and write **RTM_ADD** messages.

- Receiving an **RTM_ADD** Message

  Receiving this message indicates that some agent has attempted to add a new route to the routing table. The rt_msghdr.rtm_errno field indicates whether the attempt succeeded or failed.

  If the route just added is a new interface route, an **RTM_NEWADDR** message is also received. Multicast address additions are also reported through these messages.

- Writing an **RTM_ADD** Message

  Writing an **RTM_ADD** message adds a new route to the routing table if the message is well formed and at least one of the following elements distinguishes the new route from a route already existing in the table:

  - destination address
  - netmask value (or IPv6 prefix value)
  - gateway
  - protocol ID
  - TOS

To monitor the success of an add request under the standard message set, check the errno value, which is available from the send results.

The header for an **RTM_ADD** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_DELETE

You can both read and write **RTM_DELETE** messages.

- Receiving an **RTM_DELETE** Message

  Receiving this message indicates that some agent has tried to delete a route from the routing table. To know whether the attempt succeeded, check the rtm_errno field in the message header. If the deleted route is an interface route, this **RTM_DELETE** is followed by an **RTM_DELADDR**. The network stack also reports multicast address deletions through these messages.

  Writing an **RTM_DELETE** Message

  Writing an **RTM_DELETE** message deletes a route from the routing table if the message is well formed and if it matches a route in the table. The criteria by which a match is made are:

  - destination address
  - netmask value (or IPv6 prefix value)
  - gateway (a value of NULL functions as a wildcard)
  - protocol ID (a value of 0 functions as a wildcard)
  - TOS (a value of -1 functions as a wildcard)

  You need to specify a protocol ID, a TOS value, and a gateway value only if you want to delete a secondary route. To delete the primary route, it is enough to specify only the destination address and netmask value.

  To check the success of a delete request, monitor the socket for an incoming **RTM_DELETE**.

  The header for an **RTM_DELETE** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_CHANGE

You can both read and write **RTM_CHANGE** messages.

- Receiving an **RTM_CHANGE**

  Message Receiving this message indicates that an agent has changed the gateway, metrics, or some other property associated with the route to the specified destination.

- Writing an **RTM_CHANGE** Message

  Writing this message lets you change the gateway or metrics associated with the route to the specified destination. This route is identified by:

  - destination address
  - netmask value (or IPv6 prefix value)
  - gateway (a value of NULL does not function as a wildcard)
  - protocol ID (a value of 0 functions as a wildcard)
  - TOS (a value of -1 functions as a wildcard)

  You need to specify a protocol ID and TOS value only if you want to modify a particular route. You can set these to wildcard values if matching on them is not important to you. If multiple routes exist, you must specify a gateway in order to select the matching entry.

  All information in the message other than the destination and netmask are interpreted as values that you want written into the existing route entry. If only one route exists for a destination address and netmask, you can change the gateway by providing a new gateway value. If multiple routes exist, then you cannot change the gateway because the gateway is used to select the route that is the entry you want changed.

  The header for an **RTM_CHANGE** message is described by an rt_msghdr structure, which is defined in **net/route.h**.

RTM_GET

You can both read and write **RTM_GET** messages. Writing this message checks the route table for a route to the specified destination. To check the success of the request, monitor the socket for an incoming **RTM_GET** message. If the request was successful, the route information is appended to the message header. If the request failed to find a matching route, this is indicated in the rtm_errno field of the message header.

The criteria by which a match is made are:

WNDRVR

- destination address
- netmask value (or IPv6 prefix value)
- gateway (a value of NULL functions as a wildcard)
- protocol ID (a value of 0 functions as a wildcard)
- TOS (a value of -1 functions as a wildcard)

You need to specify the protocol ID, the TOS value, and the gateway value if you want to retrieve a secondary route. To retrieve the primary route, you need to specify only the destination address, and optionally, the netmask (or prefix) value. If the netmask value is not specified, the code does a longest-match lookup of the destination, as it would if sending a packet, rather than searching for an exact match with a specified destination-netmask pair.

The header for an **RTM_GET** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_LOSING

This is a read-only message. It indicates that the specified route might not still be valid. TCP originates this message after four or more consecutive failed retransmissions over the route.

The header for an **RTM_LOSING** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_REDIRECT

This is a read-only message. It indicates that ICMP has sent a redirect for the specified destination, the address is appended to the header in the **RTA_DST** location. The address of a preferred gateway is appended to the message in the **RTA_GATEWAY** location. The address of the author for the redirect message is appended to the header in the **RTA_AUTHOR** location.

To know whether this message indicates a change in the contents of the route table, check the rtm_errno value in the message header. A nonzero rtm_errno value indicates a failed **RTM_REDIRECT**. A zero indicates a successful **RTM_REDIRECT**. In this case, the table contains a new host-specific route. This new host route is created only if the original misdirected route was a non-host (or network) route. Otherwise, that existing host route is modified.

The header for an **RTM_REDIRECT** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_MISS

This is a read-only message. It indicates that some agent's search of the route table failed to find a route to the specified destination. The header for an **RTM_MISS** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_LOCK

You can both read and write **RTM_LOCK** messages. Receiving this message indicates that the metrics associated with the specified route are now locked. Writing this message locks the metrics for the specified route. The header for an **RTM_LOCK** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_RESOLVE

This is a read-only message. Receiving this message indicates a request to resolve a destination address to a link-layer address. The header for an **RTM_RESOLVE** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_NEWADDR

This is a read-only message. This message is one of two messages associated with adding an IP address to an interface. The other, an **RTM_ADD**, describes the route through that interface to the subnet associated with the address.

For an **RTM_NEWADDR** message, the message header is described by the ifa_msghdr structure, which is defined in **if.h**.

RTM_DELADDR

This is a read-only message. This message is the second of two messages associated with deleting an IP address from an interface. The first, an **RTM_DELETE**, describes the now deleted route through that interface to the subnet with the associated address.

WNDRVR

For an **RTM_DELADDR** message, the message header is described by the ifa_msghdr structure, which is defined in **route.h**.

RTM_IFINFO

This is a read-only message. This message reports a change in status for the specified interface. All routes through that interface are affected by the change. For an **RTM_IFINFO** message, the message header is described by an if_msghdr structure, which is defined in **if.h**.

RTM_IFANNOUNCE

This is a read-only message. It announces the arrival or departure of a network interface. The header for an **RTM_IFANNOUNCE** message is described by an if_announcemsghdr structure, which is defined in **if.h**.

Previous topic: Case/Switch Processing for Received Messages on page 33
Next topic: Extended Messages for Virtual Routing Reference on page 38

6. Extended Messages for Virtual Routing Reference

A new virtual router can be created and destroyed in various ways.

For more information see the VxWorks 7 Network Stack Programmer's Guide.

RTM_NEWVR

You can both read and write **RTM_NEWVR** messages. This message adds a new virtual routing domain. The header for an **RTM_NEWVR** message is described by an rt_msghdr structure, which is defined in **route.h**.

RTM_DELVR

You can both read and write **RTM_DELVR** messages. This message deletes a virtual routing domain. The default route table (for VR 0) cannot be removed. The header for an **RTM_DELVR** message is described by an rt_msghdr structure, which is defined in **route.h**.

Previous topic: Types of Routing Socket Messages on page 34
Next topic: RTF Flags Reference on page 38

7. RTF Flags Reference

The RTF flags indicate a route's type and other attributes.

The following table provides a complete listing of the supported **RTF_name** flags.

Table 1. RTF Flags

Flag	Description
RTF_UP	Route is up and usable.
RTF_GATEWAY	Destination is reachable through a gateway.
RTF_HOST	Route to a host.

WNDRVR

Flag	Description
RTF_REJECT	Route is administratively not reachable. IP datagrams matching this route generate an ICMP/ICMPv6 host unreachable message with code **PROHIBITED_NET** or net unreachable with code **PROHIBITED_HOST**.
RTF_DYNAMIC	Created dynamically (by redirect).
RTF_MODIFIED	Modified dynamically (by redirect).
RTF_DONE	Message confirmed (applies to routing sockets). The routing socket message has been processed.
RTF_CLONING	Generate new routes on use. This flag is usually set internally for directly connected network routes. This causes ARP entries to be cloned.
RTF_XRESOLVE	External daemon resolves name. This specifies that a cloned route be passed to a user daemon through an **RTM_MISS** message for name resolution.
RTF_LLINFO	Route contains link layer information generated by link layer (for example, ARP).
RTF_STATIC	Manually added (through **routec( )**, for example).
RTF_BLACKHOLE	Just discard packets (during updates).
RTF_PROTO2	Protocol-specific routing flag.
RTF_PROTO1	Protocol-specific routing flag.
RTF_PREF	Ignore Equal Cost Multipath and always select this entry.
RTF_MBINDING	This route is part of a multiple binding flow and unicast packets may be sent to multiple destinations. Used mainly by Mobile IP.
RTF_SKIP	A hit on this route entry should be treated as a lookup failure. Used mainly by policy routing to move on to the next policy routing rule.
RTF_LOCAL	Route represents a local address.

Previous topic: Extended Messages for Virtual Routing Reference on page 38
Next topic: Extracting Information from a Routing Socket Message on page 39

8. Extracting Information from a Routing Socket Message

A routing socket message includes all the information needed to identify a route table entry.

This includes information such as the destination address, netmask, and gateway (which are appended after the message header) as well as the route metrics (which are supplied in the members of the message header). After the header, a routing socket message can contain up to eight socket address structures.

Parsing the Routing Socket Message after the Header

If all eight socket address structures were present in a message, the structures would follow the header in the order shown below:

1. destination address
2. gateway address
3. netmask
4. cloning mask
5. interface name
6. interface address
7. address of the author of a redirect
8. broadcast or point-to-point destination address

These appended socket address structures supply all the remaining information needed to identify a route table entry for the events (add, change, delete, and so on) associated with that route.

To tell you which addresses are actually included and which are omitted, the routing socket message header provides an 8-bit mask. The name of the field that contains this bit-mask is of the form: structure_addrs, where structure is rtm, ifm, ifmam, or ifam. Each bit in the mask corresponds to one of the eight addresses listed above. If the bit is set, the address is present. If the bit is clear, the address is omitted.

Consider a bit-mask of 0x87, which is 1000 0111 in base 2. In this mask, only four bits are set, which tells you that the message after the header contains only four addresses. Because the lowest order bit is set, you know that the first address after the header is a destination address. Because the second and third lowest order bits are set, you know that the second address is a gateway address, which is followed by a netmask. The only other set bit is the highest order bit, which tells you that the final (fourth address) is a broadcast address.

The **route.h** file defines a set of constants for the bits in a structure_addrs mask. You can AND these constants against structure_addrs to test whether a particular address is present in the message. The constants for rtm_addrs are:

```
#define RTA_DST         0x1     /* destination sockaddr present */
#define RTA_GATEWAY     0x2     /* gateway sockaddr present */
#define RTA_NETMASK     0x4     /* netmask sockaddr present */
#define RTA_GENMASK     0x8     /* cloning mask sockaddr present */
#define RTA_IFP         0x10    /* interface name sockaddr present */
#define RTA_IFA         0x20    /* interface addr sockaddr present */
#define RTA_AUTHOR      0x40    /* sockaddr for author of redirect */
#define RTA_BRD         0x80    /* for NEWADDR, broadcast or p-p dest addr */
```

The **RTAX_MAX** constant is defined in **route.h** as 8, the maximum number of socket address structure types included in a routing socket message. The constants for indexing into an array of pointers to sockaddr structures are also defined. These constants are:

```
#define RTAX_DST        0       /* destination sockaddr present */
#define RTAX_GATEWAY    1       /* gateway sockaddr present */
#define RTAX_NETMASK    2       /* netmask sockaddr present */
#define RTAX_GENMASK    3       /* cloning mask sockaddr present */
#define RTAX_IFP        4       /* interface name sockaddr present */
#define RTAX_IFA        5       /* interface addr sockaddr present */
#define RTAX_AUTHOR     6       /* sockaddr for author of redirect */
```


WNDRVR

```
#define RTAX_BRD        7       /* for NEWADDR, broadcast or p-p dest addr */
#define RTAX_MAX        8       /* size of array to allocate */
```

Previous topic: RTF Flags Reference on page 38
Next topic: Building a Routing Socket Message on page 41

9. Building a Routing Socket Message

The messages you can transmit over a routing socket are restricted to a few basic types.

For details see Types of Routing Socket Messages on page 34.

Within these messages, you specify a route table entry using the following values:

- a destination address, in the **RTAX_DST** socket address structure
- a netmask, in the **RTAX_NETMASK** socket address structure
- a gateway address, in the **RTAX_GATEWAY** socket address structure
- a TOS value, in the **RTAX_DST** socket address structure (optional, policy routing)
- a protocol ID, in the **RTAX_DST** socket address structure (optional, policy routing)

You must append the socket address structures referred to above to the routing socket message header.

The address for the socket address structures in a routing message must start on an address evenly dividable by 4.

To specify route metrics in the routing socket message, use the rt_metrics structure in the rtm_rmx field of the routing socket message header. For more information, see the discussion of *rtm_rmx – optional* in Working with Internet Addresses on page 27 for information on setting metrics .

Note: Depending on the message type and whether the table stores more than one entry for a destination, you may need to specify a destination address only, or a destination and netmask only, or a gateway value.

Previous topic: Extracting Information from a Routing Socket Message on page 39
Next topic: Header Structure Field Values Reference on page 41

10. Header Structure Field Values Reference

Each of the writable routing socket message types use an rt_msghdr structure for its header.

When building a routing socket message, fill in the rt_msghdr members as follows:

rtm_msglen

This field is required in an outgoing message.

This field expects the length of the entire socket message. The reported size includes the header plus all appended socket addressees. The best time to assign this value is after you have appended the last address to the routing socket message. See below, **rtm_addrs**.

rtm_version

This field is required in an outgoing message.

This field expects the version ID of the routing socket message. The current version is 4.

rtm_type

This field is required in an outgoing message.

This field expects a value indicating the routing socket message type RTM_*type*.

These message types (described in Types of Routing Socket Messages on page 34) let you add, change, delete, get, or lock an entry in the route table. The response to an add, change, delete, get, or lock message is an incoming add, change, lock, get, or delete message on your socket. You can determine the success or failure of your request by reading the rtm_errno field in the header of the response message.

rtm_index

This field is read-only, ignored in outgoing messages.

The index value for the associated network interface.

rtm_flags

This field is for add and delete messages only.

In an outgoing message, this field expects an integer value whose bits describe the route. This value includes bits that indicate whether the interface is online or offline, whether this is a host route or a gateway route, and more. For a complete listing of valid flags, see RTF Flags Reference on page 38.

rtm_addrs

This field is required in an outgoing message.

This field expects a bit mask in which you have set bits that identify which addresses you have appended to the end of this message.

rtm_pid

This field is optional in an outgoing message.

A process or task ID or some other identifier for yourself. You can use this to recognize responses to the commands you have issued.

rtm_seq

This field is optional in an outgoing message.

This field expects the sequence number that you want to assign to this command. This value is returned in the response message. Use it in conjunction with **rtm_pid** to help distinguish the response to this command from responses to other commands.

rtm_errno

This field is read-only in an outgoing message.

Read this value in the response to a command. If its value is zero, the command was successful. Any other value indicates an error.

rtm_use

This field is read-only in an outgoing message.

This is strictly an output vehicle. It tells you the number of times that the specified route was used.

rtm_inits

This field is optional, for setting metrics.

This field expects a value whose bits indicate which metrics this message would initialize. Use the **IPNET_RTV_** constants defined in

installDir **/vxworks-7/pkgs_v2/net/ipnet**x.x.x.x **/coreip**-x.x.x.x **/src/ipnet2/include/ipnet.h**

to help set bits for this field.

rtm_inits can be set to ~0 (all bits set to one), if all metric fields should be affected. It can be set to 0 (all bits set to zero), if no metric fields should be affected. Setting **rtm_inits** to zero during an add operation results in the stack choosing the values for the metric.

rtm_rmx

This field is optional, for setting metrics.

This field expects an rt_metrics structure, defined in **route.h**. If you use this structure to specify metric values for a route, you must also set the appropriate flags in the **rtm_inits** field.

Previous topic: Building a Routing Socket Message on page 41
Next topic: Sample Routing Socket Messages Reference on page 43

11. Sample Routing Socket Messages Reference

The following examples illustrate the use of routing socket messages to find a gateway and add an address to a route message.

Finding a Gateway

```
#include <arpa/inet.h>
#include <errno.h>
#include <net/route.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>

/*
 * Functions defines as "static" are helper functions that are used in
 * one or more of the examples
 */

/*
 * Given an AF_ROUTE message, initializes an array of socket address
 * pointer so that addrs[RTAX_DST] would return a pointer to the
 * destination address, addrs[RTAX_GATEWAY] would return a pointer to
 * the gateway, or NULL if no gateway is present in the message.
 *
 * addrs - array of pointers to socket addresses. Must point to an
 *         area that can hold at least RTAX_MAX pointers.
 * rtm - pointer to a route message.
```

```
 */
static void
fillAddrs(struct sockaddr *addrs[RTAX_MAX],
                  struct rt_msghdr *rtm)
{
        int              rta_index;
        struct sockaddr *sa;

        sa = (struct sockaddr *) (rtm + 1);

        for (rta_index = 0; rta_index < RTAX_MAX; rta_index++)
        {
                if (rtm->rtm_addrs & (1 << rta_index))
                {
                        /*
                         * Address present in message
                         */
                        addrs[rta_index] = sa;

                        /*
                         * Move socket address pointer to next address.
                         */
                        sa = (struct sockaddr *) ((char *) sa + sa->sa_len);
                }
                else
                        /*
                         * This address is not present
                         */
                        addrs[rta_index] = NULL;
        }
}
```

Adding an IPv4 Address to a Route Message

This example adds an IPv4 address to a route message. It requires the same include files as those used in *Finding a Gateway*, above.

```
/*
 * Adds an IPv4 address to a route message. The buffer where the route
 * message is stored must be large enough to hold the address.
 *
 * rtm - pointer to the route message.
 * rtm_addr - RTA_xxx constant that defines what type of address being
 *            pushed
 * in_addr_str - IPv4 address in presentation format.
 */
static void
pushInetAddr(struct rt_msghdr *rtm,
                         int rtm_addr,
                         const char *in_addr_str)
{
        struct sockaddr_in *sin;

        sin = (struct sockaddr_in *) ((char *) rtm + rtm->rtm_msglen);

        memset(sin, 0, sizeof(struct sockaddr_in));
```

```
        sin->sin_family = AF_INET;
        sin->sin_len = sizeof(struct sockaddr_in);
        inet_pton(AF_INET, in_addr_str, &sin->sin_addr);

        rtm->rtm_addrs  |= rtm_addr;
        rtm->rtm_msglen += sin->sin_len;
}
```

Finding the Address of the Next Hop Node

This example finds the address of the next hop node, using the destination address. It requires the same include files as those used in *Finding a Gateway*, above.

```
 /* Returns the address of the next hop node given a destination
 * address.
 *
 * dst_str - address of the destination for which the next hop node
 *           should be returned.
 *
 */
int
nexthopShow(const char *dst_str)
{
        int rsock;
        struct rt_msghdr *rtm;
        char buf[sizeof(struct rt_msghdr) + RTAX_MAX * sizeof(struct sockaddr)];
        pid_t pid = taskIdSelf();
        int seq = rand();
        int err;

        rsock = socket(AF_ROUTE, SOCK_RAW, 0);
        if (rsock < 0)
                goto errout;

        /*
         * Message format is:
         * < rtm | dst >
         * The length field of in 'rtm' is the length of all structures
         * between '<' and '>'
         */
        rtm = (struct rt_msghdr *) buf;
        memset(rtm, 0, sizeof(*rtm));
        rtm->rtm_msglen  = sizeof(*rtm);
        rtm->rtm_version = 4;
        rtm->rtm_type    = RTM_GET;
        rtm->rtm_pid     = pid;
        rtm->rtm_seq     = seq;
        rtm->rtm_table   = 254;

        pushInetAddr(rtm, RTA_DST, dst_str);

        /*
         * Send the message to the kernel
         */
        if (send(rsock, (void *) rtm, sizeof(buf), 0) < 0)
                goto errout;
```

```
        /*
         * Receive message until we get a response to the message we sent.
         * That response is identified by having the same "pid"
         * and sequence number as our request and a type of RTM_ADD
         */
        while (recv(rsock, (void *) rtm, sizeof(buf), 0) > 0)
        {
                if (rtm->rtm_type == RTM_GET
                        && rtm->rtm_pid == pid
                        && rtm->rtm_seq == seq)
                {
                        if (rtm->rtm_errno == 0)
                        {
                                char nexthop_str[INET_ADDRSTRLEN];
                                struct sockaddr_in *addrs[RTAX_MAX];
                                /*
                                 * Operation was successful
                                 */
                                fillAddrs((struct sockaddr **) addrs, rtm);
                                if (rtm->rtm_flags & RTF_GATEWAY)
                                        /*
                                         * Reachable through a gateway
                                         */
                                        inet_ntop(AF_INET,
                                                          &addrs[RTAX_GATEWAY]->sin_addr,
                                                          nexthop_str,
                                                          sizeof(nexthop_str));
                                else
                                        /*
                                         * Destination directly reachable
                                         */
                                        strncpy(nexthop_str,
                                                        dst_str,
                                                        sizeof(nexthop_str));

                                printf("%s\n", nexthop_str);
                        }

                        close(rsock);
                        return rtm->rtm_errno;
                }
        }

 errout:
        err = errno;
        close(rsock);
        return err;
}
```

Previous topic: Header Structure Field Values Reference on page 41

8. NETLINK AND RTNETLINK SOCKETS

1. Introduction

You can add netlink socket support to VxWorks, set up netlink sockets and transmit data on them.

Overview

Netlink is a network service that enables communication between the kernel and user-space processes.

In many systems, only the most essential code is placed in the kernel. The goal of this practice is to simplify the development and maintenance of the kernel while enhancing system stability by placing application code in user space. To support this practice, however, application developers must find a way to enable communication between the kernel and user-space processes.

The Wind River Network Stack uses netlink to enable a bidirectional communication link between the kernel and user-space processes. The bidirectional, or duplex, property means that either a user-space process or the kernel module can initiate communication. Netlink is implemented using a standard sockets-based interface for user processes and an internal kernel API for kernel modules.

Rtnetlink is a protocol extension to netlink that implements management and information retrieval for links, addresses, rules, routes, and neighbors. It is the protocol used by Linux and Linux applications to handle network-related protocols and subsystems.

Netlink and Rtnetlink

The Wind River Network Stack implements netlink using sockets and an internal kernel API. Netlink sockets are similar to TCP/IP sockets, but instead of using **AF_INET** as the domain, netlink sockets use **AF_NETLINK**. Both **SOCK_RAW** and **SOCK_DGRAM** are valid values for socket_type, but the netlink protocol does not distinguish between datagram and raw sockets. The parameter netlink_family selects the kernel module or netlink group to communicate with.

The netlink family currently implemented in the Wind River Network Stack is **IP_NETLINK_ROUTE**. This family receives routing and link updates and may be used to modify the routing tables (both IPv4 and IPv6), IP addresses, link parameters, neighbor setups, queueing disciplines, traffic classes, and packet classifiers. Use this family to enable communication between user-space routing daemons such as RIP and the kernel packet forwarding module. You can enable additional protocols by defining them in the following file:

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/ipcom/include/ipcom_netlink.h**

Configuring VxWorks for Netlink and Rtnetlink Sockets

To add the netlink socket functionality to your project, see Source Build Configuration Options (VSB) Reference on page 5 and Kernel Configuration Components (VIP) Reference on page 6.

The **INCLUDE_IPNET_USE_NETLINKSOCK** component includes the routing socket interface and implements both netlink and rtnetlink. If you create an IPv6 network stack, the IPv6 features of rtnetlink are automatically enabled. No further implementation is necessary.

There are no configuration parameters or externally callable functions directly associated with this component.

Next topic: Setting up a Netlink Socket on page 48

2. Setting up a Netlink Socket

To set up a netlink socket, use an socket( ) call.

About This Task

After you create the socket, you can configure its options.

The following example illustrates how to create, bind and terminate a Netlink socket.

Procedure

1. Create a netlink socket.

   Call socket( ) with **AF_NETLINK** as the domain, **SOCK_RAW** as the type, and **IP_NETLINK_ROUTE** as *netlink_family*

   For example:

   ```
   socket(AF_NETLINK, SOCK_RAW,IP_NETLINK_ROUTE)
   ```

2. Bind a netlink socket.

   For example:

   ```
   struct Ip_sockaddr_nl nladdr;
   ipcom_memset(&nladdr, 0, sizeof(nladdr));
   nladdr.nl_family = AF_NETLINK;
   nladdr.nl_len = sizeof(nladdr);
   /* If you want to listen to MULTICAST groups; that is subscribe to various
   network events, specify groups, in this case L2 events */
   nladdr.nl_groups = (1 << (IP_RTMGRP_LINK - 1))
   bind(nlsock, (struct sockaddr*) &nladdr, sizeof(nladdr));
   ```

3. Terminate data on a Netlink socket.

   Call sendto( ), recvfrom( ), sendmsg( ), or recvmsg( ) as appropriate to send or receive data on a netlink socket.

   You can also call send( ), but if you do, you must first call connect( ) before connecting the socket.

Previous topic: Introduction on page 47
Next topic: Working with Netlink Messages on page 48

3. Working with Netlink Messages

Netlink messages consist of a byte stream with one or multiple nlmsghdr headers and associated payload. These messages are aligned, with the precise alignment varying by architecture.

To create messages with the correct alignment, use only the standard **IP_NLMSG_*** macros when accessing the byte stream.

Previous topic: Setting up a Netlink Socket on page 48

WNDRVR

Next topic: Sending Netlink Messages on page 49

4. Sending Netlink Messages

In multipart messages (multiple nlmsghdr headers with associated payload in one byte stream), the first and all following headers have the **IP_NLM_F_MULTI** flag set, except for the last header, which has the type **IP_NLMSG_DONE**.

After each nlmsghdr, the payload follows.

```
struct Ip_nlmsghdr {
    __u32 nlmsg_len;   /* Length of message including header. */
    __u16 nlmsg_type;  /* Type of message content. */
    __u16 nlmsg_flags; /* Additional flags. */
    __u32 nlmsg_seq;   /* Sequence number. */
    __u32 nlmsg_pid;   /* PID of the sending process. */
};
```

The netlink socket also requires a message header to provide a common ground for netlink messages of all protocol types. Because the kernel netlink core assumes the existence of the preceding header in each netlink message, an application must supply this header in each netlink message it sends.

nlmsg_len specifies the total length of the netlink message, including the header. It is required by netlink core. nlmsg_flags is used to give additional control to a message; it is read and updated by netlink core. nlmsg_seq and nlmsg_pid are used by applications to track the message, and they are also opaque to netlink core.

A netlink message thus consists of nlmsghdr and the message payload. Once a message has been entered, it enters a buffer pointed to by the nlh pointer.

nlmsg_type can be used by applications and is an opaque value to netlink core. It can be one of the standard message types. **IP_NLMSG_NOOP** message is to be ignored. **IP_NLMSG_ERROR** message signals an error and the payload contains an Ip_nlmsgerr structure. **IP_NLMSG_DONE** message terminates a multipart message.

```
struct Ip_nlmsgerr {
    int error; /* Negative errno or 0 for acknowledgements */
    struct Ip_nlmsghdr msg; /* Message header that caused the error */
};
```

After completing these structures, call sendmsg( ) to transmit the message.

Previous topic: Working with Netlink Messages on page 48
Next topic: Receiving Netlink Messages on page 49

5. Receiving Netlink Messages

A receiving application must allocate a buffer large enough to hold the netlink message headers and message payloads.

It fills the structure Ip_msghdr msg and uses an recvmsg( ) call to receive the netlink message, assuming the buffer is pointed to by *nlh* . For example:

WNDRVR

```
struct Ip_sockaddr_nl nladdr; int val; struct Ip_iovec vec; struct Ip_msghdr
msg; struct Ip_nlmsghdr *nlh; char rbuf[512];
ipcom_memset(&msg, 0, sizeof(msg));
msg.msg_name = &nladdr;
msg.msg_namelen = sizeof(nladdr);
msg.msg_iov = &vec;
msg.msg_iovlen = 1;
vec.iov_base = rbuf;
vec.iov_len = sizeof(rbuf);
if ((val = ipcom_recvmsg(nlhandle->nlsock, &msg, 0)) < 0)
    return -ipcom_errno;
nlh = (struct Ip_nlmsghdr*)rbuf;
while (IP_NLMSG_OK(nlh, (unsigned)val)) {
    nlh = IP_NLMSG_NEXT(nlh, val);
}
return 0;
```

After the message has been received correctly, *nlh* should point to the header of the netlink message just received. *nladdr* should contain the destination address of the received message, which consists of the PID and the multicast groups to which the message is sent. The macro **IP_NLMSG_DATA(NLH)** returns a pointer to the payload of the netlink message. When the message has been received, close the socket by calling socketclose( ).

Previous topic: Sending Netlink Messages on page 49
Next topic: Limitations of the Netlink Protocol on page 50

6. Limitations of the Netlink Protocol

Netlink is not a reliable protocol. It attempts to deliver a message to the specified destinations, but it may drop messages when an out-of-memory condition or other error occurs.

For reliable transfer the sender can request an acknowledgement from the receiver by setting the **IP_NLM_F_ACK** flag. An acknowledgment is an **IP_NLMSG_ERROR** packet with the error field set to 0. The application must generate acknowledgements for received messages itself. The kernel tries to send an **IP_NLMSG_ERROR** message for every failed packet. A user process should follow this convention also.

However, reliable transmissions from kernel to user are impossible in any case. The kernel cannot send a netlink message if the socket buffer is full: the message is dropped and the kernel and the user-space process no longer has the same view of kernel state. It is up to the application to detect when this happens (via the **ENOBUFS** error returned by recvmsg(2)) and re-synchronize.

Previous topic: Receiving Netlink Messages on page 49
Next topic: Flag Bits Reference on page 50

7. Flag Bits Reference

Standard Flag Bits in nlmsg_flags

The standard flag bits are used to determine if a message is a request or reply. If a message is a request, the standard flag bits are also used to determine if a message is a request for a specific resource or a request for all information related to the request.

The nlmsg_flags are defined in **ipcom_netlink.h**. The following standard flag bits are available in nlmsg_flags:

IP_NLM_F_REQUEST

Must be set on all request messages.

IP_NLM_F_MULTI

The message is part of a multipart message terminated by **IP_NLMSG_DONE**.

IP_NLM_F_ACK

Request for an acknowledgment on success.

IP_NLM_F_ECHO

Echo this request.

Additional Flag Bits for GET Requests

GET requests such as **IP_RTM_GETROUTE** and **IP_RTM_GETRULE** request that the rtnetlink API respond with information related to the request. You may specify a GET request as either a specific request or a “wildcard” request.

For example, in a specific request, you might ask the rtnetlink API to provide attributes related to that particular family, identifying a resource you want to receive information about. In a "wildcard" GET request, you might specify the **IP_NLM_F_DUMP** flag, which gets information on all resources related to the GET request (such as all routes, all rules, all neighbors, etc).

The nlmsg_flags are defined in **ipcom_netlink.h**. The following additional flag bits are available for GET requests:

IP_NLM_F_ROOT

Return the complete table instead of a single entry.

IP_NLM_F_ATOMIC

Return an atomic snapshot of the table.

IP_NLM_F_DUMP

Convenience macro; equivalent to (**IP_NLM_F_ROOT** or **IP_NLM_F_MATCH**).

Additional Flag Bits for NEW Requests

NEW requests such as **IP_RTM_NEWROUTE** or **IP_RTM_NEWRULE** install or create a new instance of a resource in the IPNET2 kernel.

The nlmsg_flags are defined in **ipcom_netlink.h**. The following additional flag bits are available for NEW requests:

IP_NLM_F_REPLACE

Replace existing matching object.

IP_NLM_F_EXCL

Do not replace if the object already exists.

IP_NLM_F_CREATE

Create object if it doesn't already exist.

IP_NLM_F_APPEND

Add to the end of the object list.

WNDRVR

nlmsg_seq and nlmsg_pid are used to track messages. nlmsg_pid shows the origin of the message. Note that there is not a 1:1 relationship between nlmsg_pid and the PID of the process if the message originated from a netlink socket. For further information on setting the process ID, see Address Formats on page 52.

Both nlmsg_seq and nlmsg_pid are opaque to netlink core. In other words, the kernel sets the same values in the reply that were used in the request. This method provides a way for the user-space process to match a reply with the original request.

Previous topic: Limitations of the Netlink Protocol on page 50
Next topic: Address Formats on page 52

8. Address Formats

The structure definition for the Ip_sockaddr_nl address type is in *installDir* **/vxworks-7/pkgs_v2/net/ipnet-**x.x.x.x **/coreip-**x.x.x.x **/src/ipcom/include/ipcom_netlink.h**.

The sockaddr_nl structure describes a netlink client in user space or in the kernel. A sockaddr_nl can be either unicast (only sent to one peer) or sent to netlink multicast groups (nl_groups not equal to 0).

To send a netlink message to the kernel or another user-space process, you must supply another sockaddr_nl structure as the destination address, just as you would if you were sending a UDP packet with sendmsg( ).

nl_pid is the unicast address of the netlink socket. If the message is sent to the kernel, both the nl_pid and nl_groups structures should be supplied with 0. If the message is a unicast message addressed to another process, nl_pid is the PID of the other process and nl_groups is 0.

For a user-space process, nl_pid is usually the PID of the process owning the destination socket. However, nl_pid identifies a netlink socket, not a process. If a process owns several netlink sockets, then nl_pid can only be equal to the process ID for at most one socket.

There are two ways to assign nl_pid to a netlink socket. If the application sets nl_pid before calling bind( ), the application must also ensure that nl_pid is unique. If the application sets it to 0, the kernel must assign it. The kernel assigns the process ID to the first netlink socket the process opens and assigns a unique nl_pid to every netlink socket that the process subsequently creates.

nl_groups is a bitmask with every bit representing a netlink group number. Each netlink family has a set of 32 multicast groups. When bind( ) is called on the socket, the nl_groups field should be set to a bitmask of the groups to which it should listen. The default value for this field is 0, which means that no multicasts are received.

A socket may multicast messages to any of the multicast groups by setting nl_groups to a bitmask of the groups it wishes to send to when it calls sendmsg( ) or does a connect( ). Any replies to a message received for a multicast group should be sent back to the sending PID and the multicast group.

Previous topic: Flag Bits Reference on page 50
Next topic: Macros Reference on page 52

9. Macros Reference

You can use macros defined in **ipcom_netlink.h** when creating netlink messages.

IP_NLMSG_ALIGN

Round the length of a netlink message up to align it properly.

IP_NLMSG_LENGTH

Given the payload length, len, this macro returns the aligned length to store in the nlmsg_len field of the Ip_nlmsghdr.

IP_NLMSG_SPACE

Return the number of bytes that a netlink message with payload of len would occupy.

IP_NLMSG_DATA

Return a pointer to the payload associated with the passed Ip_nlmsghdr.

IP_NLMSG_NEXT

Get the next Ip_nlmsghdr in a multipart message. The caller must check if the current Ip_nlmsghdr didn't have the **IP_NLMSG_DONE** set—this function doesn't return NULL on end. The length parameter is an lvalue containing the remaining length of the message buffer. This macro decrements it by the length of the message header.

IP_NLMSG_OK

Return true if the netlink message is not truncated and OK to parse.

IP_NLMSG_PAYLOAD

Return the length of the payload associated with the Ip_nlmsghdr.

Previous topic: Address Formats on page 52
Next topic: Working with Rtnetlink Sockets on page 53

10. Working with Rtnetlink Sockets

Rtnetlink is a protocol extension to netlink. Depending on the particular features enabled in your network stack, it implements management and information retrieval for the link layer, addresses, rules, virtual routing, policy routing, and ARP or NDP.

Netlink Route Sockets

Use **IP_NETLINK_ROUTE** sockets to control network routes, IP addresses, link parameters, neighbor setups, queueing disciplines, traffic classes, and packet classifiers. These sockets are based on netlink messages.

Interfaces

Rtnetlink consists of the following interfaces:

RTNETLINK LINK

The **LINK** interface lets you add or remove supported interfaces (VLAN). It also lets you make administrative changes (up, down) and MTU changes.

RTNETLINK ADDRESS

The **ADDRESS** interface lets you add or remove IPv4 or IPv6 addresses to various interfaces.

RTNETLINK ROUTE

The **ROUTE** interface lets you add or remove IPv4 or IPv6 routes.

RTNETLINK RULE

The **RULE** interface lets you add or remove IPv4 or IPv6 policy rules, which decide how traffic should be routed.

RTNETLINK NEIGH

The **NEIGH** interface lets you add or remove ARP entries for IPv4 or NDP entries for IPv6.

RTNETLINK TUNNEL

The **TUNNEL** interface lets you add, remove, or configure IPv4 or IPv6 tunnels.

Rtnetlink Routing Attributes

Some rtnetlink messages have optional attributes after the initial header:

```
struct Ip_rtattr {
    unsigned short rta_len; /* Length of option */
    unsigned short rta_type; /* Type of option */
    /* Data follows */
};
```

These attributes should be only manipulated using the **IP_RTA**_* macros or **ipcom_netlink.h**.

Working with Rtnetlink Messages

Rtnetlink consists of these message types (in addition to standard netlink messages):

- **IP_RTM_NEWLINK**, **IP_RTM_DELLINK**, **IP_RTM_GETLINK**
- **IP_RTM_NEWADDR**, **IP_RTM_DELADDR**, **IP_RTM_GETADDR**
- **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, **IP_RTM_GETROUTE**
- **IP_RTM_NEWRULE**, **IP_RTM_DELRULE**, **IP_RTM_GETRULE**

IP_RTM_NEWLINK, **IP_RTM_DELLINK**, **IP_RTM_GETLINK**

Use the **IP_RTM_NEWLINK**, **IP_RTM_DELLINK**, or **IP_RTM_GETLINK** message type to create, remove, or get information about a specific network interface. These messages contain an Ip_ifinfomsg structure followed by a series of rtattr structures.

```
struct Ip_ifinfomsg
{
        Ip_u8   ifi_family;
        Ip_u8   __ifi_pad;
        Ip_u16  ifi_type;                /* ARPHRD_* */
        Ip_s32  ifi_index;              /* Link index   */
        Ip_u32  ifi_flags;              /* IFF_* flags  */
        Ip_u32  ifi_change;         /* IFF_* change mask */
};
```

ifi_flags contains the device flags. See the netdevice 7 man page for further information. ifi_index is the unique interface index. ifi_change is reserved for future use and should be always set to 0xFFFFFFFF.

WNDRVR

Table 1. Routing Attributes for IP_RTM_NEWLINK, IP_RTM_DELLINK, IP_RTM_GETLINK Messages

rta_type	Value Type	Description
IP_IFLA_UNSPEC	-	Unspecified
IP_IFLA_ADDRESS	Hardware address	Interface L2 address
IP_IFLA_BROADCAST	Hardware address	L2 broadcast address
IP_IFLA_IFNAME	ASCIIZ string	Device name
IP_IFLA_MTU	Unsigned int	MTU of the device
IP_IFLA_LINK	int	Link type
IP_IFLA_QDISC	ASCIIZ string	Queueing discipline
IP_IFLA_STATS	struct net_device_stats	Interface statistics

IP_RTM_NEWADDR, **IP_RTM_DELADDR**, **IP_RTM_GETADDR**

Use the **IP_RTM_NEWADDR**, **IP_RTM_DELADDR**, or **IP_RTM_GETADDR** message type to add, remove, or receive information about an IP address associated with an interface. These messages contain an Ip_ifaddrmsg structure, optionally followed by rtaddr routing attributes.

```
struct Ip_ifaddrmsg
{
        Ip_u8   ifa_family;
        Ip_u8   ifa_prefixlen;  /* The prefix length            */
        Ip_u8   ifa_flags;      /* Flags             */
        Ip_u8   ifa_scope;      /* See above                     */
        Ip_s32  ifa_index;      /* Link index                   */
    };
```

ifa_family is the address family type (currently **AF_INET** or **AF_INET6**). ifa_prefixlen is the length of the address mask of the address if defined for the family (like for IPv4). ifa_scope is the address scope, ifa_index is the interface index of the interface the address is associated with. ifa_flags is a flag word of **IP_IFA_F_SECONDARY** for secondary address (old alias interface), **IP_IFA_F_PERMANENT** for a permanent address set by the user, and other undocumented flags.

The following table lists the address attributes for this command family.

Table 2. Address Attributes for IP_RTM_NEWADDR, IP_RTM_DELADDR, IP_RTM_GETADDR Messages

rta_type	Value Type	Description
IP_IFA_UNSPEC	-	Unspecified
IP_IFA_ADDRESS	Raw protocol address	Interface address
IP_IFA_LOCAL	Raw protocol address	Local address

WNDRVR

rta_type	Value Type	Description
IP_IFA_LABEL	ASCIIZ string	Name of the interface
IP_IFA_BROADCAST	Raw protocol address	Broadcast address
IP_IFA_ANYCAST	Raw protocol address	Anycast address
IP_IFA_CACHEINFO	struct ifa_cacheinfo	Address information
IP_IFA_MULTICAST	Raw protocol address	Multicast address

IP_RTM_NEWROUTE, **IP_RTM_DELROUTE**, **IP_RTM_GETROUTE**

Use the **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, or **IP_RTM_GETROUTE** message type to create, remove, or receive information about a network route. These messages contain an Ip_rtmsg structure with an optional sequence of rtattr structures following. For **IP_RTM_GETROUTE**, setting rtm_dst_len and rtm_src_len to 0 means you get all entries for the specified routing table. For the other fields except rtm_table and rtm_protocol, 0 is the wildcard.

```
struct Ip_rtmsg
{
        Ip_u8           rtm_family;
        Ip_u8           rtm_dst_len;
        Ip_u8           rtm_src_len;
        Ip_u8           rtm_tos;
        Ip_u8           rtm_table;      /* Routing table id */
        Ip_u8           rtm_protocol;   /* Routing protocol; see below */
        Ip_u8           rtm_scope;      /* See below */
        Ip_u8           rtm_type;       /* See below    */
        Ip_u32          rtm_flags;
};
```

The following table lists the route types available with **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, and **IP_RTM_GETROUTE** messages.

Table 3. Route Types for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rtm_type	Route Type
IP_RTN_UNSPEC	Unknown route
IP_RTN_UNICAST	Gateway or direct route
IP_RTN_LOCAL	Local interface route
IP_RTN_BROADCAST	Local broadcast route (sent as a broadcast)
IP_RTN_ANYCAST	Local broadcast route (sent as a unicast)
IP_RTN_MULTICAST	Multicast route

rtm_type	Route Type
IP_RTN_BLACKHOLE	Packet dropping route
IP_RTN_UNREACHABLE	Unreachable destination
IP_RTN_PROHIBIT	Packet rejection route
IP_RTN_THROW	Continue routing lookup in another table
IP_RTN_NAT	Network address translation rule
IP_RTN_XRESOLVE	Use external resolver
IP_RTN_PROXY	Proxy ARP route
IP_RTN_CLONE	Cloning route

The following table defines the route origins—that is, the entity that added the route—for **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, and **IP_RTM_GETROUTE** messages.

Table 4. Route Origins for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rtm_protocol	Route Origin
IP_RTPROT_UNSPEC	Unknown
IP_RTPROT_REDIRECT	By an ICMP redirect (currently unused)
IP_RTPROT_KERNEL	By the kernel
IP_RTPROT_BOOT	During boot
IP_RTPROT_STATIC	By the administrator

Values larger than **IP_RTPROT_STATIC** are not interpreted by the kernel. They are just for user information. They may be used to tag the source of routing information or to distinguish between multiple routing daemons. See **ipcom_netlink.h** for the routing daemon identifiers which are already assigned.

The following table defines the distance to the destination.

Table 5. Distance to Destination for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rtm_scope	Distance to Destination
rtm_scope	Distance to Destination
IP_RT_SCOPE_UNIVERSE	Global route

rtm_scope	Distance to Destination
IP_RT_SCOPE_SITE	Interior route in the local autonomous system
IP_RT_SCOPE_LINK	Route on this link
IP_RT_SCOPE_HOST	Route on the local host
IP_RT_SCOPE_NOWHERE	Destination does not exist

The values between **IP_RT_SCOPE_UNIVERSE** and **IP_RT_SCOPE_SITE** are available to the user.

The following table defines flags for **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE** and **IP_RTM_GETROUTE** messages.

Table 6. Flags for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rtm_flags	Description
IP_RTM_F_NOTIFY	If the route changes, notify the user via rtnetlink.
IP_RTM_F_CLONED	Route is cloned from another route.

The following table defines the routing table options for **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, and **IP_RTM_GETROUTE** messages

Table 7. Routing Table Options for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rtm_table	Table
IP_RT_TABLE_UNSPEC	An unspecified routing table
IP_RT_TABLE_DEFAULT	The default table
IP_RT_TABLE_MAIN	The main table
IP_RT_TABLE_LOCAL	The local table

The user may assign arbitrary values between **IP_RT_TABLE_UNSPEC** and **IP_RT_TABLE_DEFAULT**.

The following table defines the routing attributes for **IP_RTM_NEWROUTE**, **IP_RTM_DELROUTE**, and **IP_RTM_GETROUTE** messages.

WNDRVR

Table 8. Routing Attributes for IP_RTM_NEWROUTE, IP_RTM_DELROUTE, IP_RTM_GETROUTE Messages

rta_type	Value Type	Description
IP_RTA_UNSPEC	-	Ignored
IP_RTA_DST	Protocol address	Route destination address
IP_RTA_SRC	Protocol address	Route source address
IP_RTA_IIF	int	Input interface index
IP_RTA_OIF	int	Output interface index
IP_RTA_GATEWAY	Protocol address	The gateway of the route
IP_RTA_PRIORITY	int	Priority of route
IP_RTA_PREFSRC	Protocol address	
IP_RTA_METRICS	int	Route metric

IP_RTM_NEWNEIGH, **IP_RTM_DELNEIGH**, **IP_RTM_GETNEIGH**

Use the **IP_RTM_NEWNEIGH**, **IP_RTM_DELNEIGH**, and **IP_RTM_GETNEIGH** message type to add, remove, or receive information about a neighbor table entry (for example, an ARP entry). The message contains an Ip_ndmsg structure.

```
struct Ip_ndmsg
{
        Ip_u8   ndm_family;
        Ip_u8   ndm_pad1;
        Ip_u16  ndm_pad2;
        Ip_s32  ndm_ifindex;    /* Link index                   */
        Ip_u16  ndm_state;
        Ip_u8   ndm_flags;
        Ip_u8   ndm_type;
};
struct nda_cacheinfo
{
        uint32_t           ndm_confirmed;
        uint32_t           ndm_used;
        uint32_t           ndm_updated;
        uint32_t           ndm_refcnt;
    };
```

The following table defines bit masks for neighbor table entries.

Table 9. Bit Masks for Neighbor Table Entries in IP_RTM_NEWNEIGH, IP_RTM_DELNEIGH, IP_RTM_GETNEIGH Messages

ndm_state	State
IP_NUD_INCOMPLETE	A currently resolving cache entry
IP_NUD_REACHABLE	A confirmed working cache entry
IP_NUD_STALE	An expired cache entry
IP_NUD_DELAY	An entry waiting for a timer
IP_NUD_PROBE	A cache entry that is currently reprobed
IP_NUD_FAILED	An invalid cache entry
IP_NUD_NOARP	A device with no destination cache
IP_NUD_PERMANENT	A static entry

The following table defines the valid ndm_flags.

Table 10. Valid ndm_flags in IP_RTM_NEWNEIGH, IP_RTM_DELNEIGH, IP_RTM_GETNEIGH Messages

ndm_flags	Valid Entry
IP_NTF_PROXY	A proxy ARP entry
IP_NTF_ROUTER	An IPv6 router

The following table defines the rta_type field in the rtaddr structure.

Table 11. Route Attributes in IP_RTM_NEWNEIGH, IP_RTM_DELNEIGH, IP_RTM_GETNEIGH Messages

rta_type	Description
IP_NDA_UNSPEC	Unknown type.
IP_NDA_DST	A neighbor cache network layer destination address. This attribute carries the layer 3 address (either IPv4 or IPv6) for which the NEIGH entry was created.
IP_NDA_LLADDR	A neighbor cache link layer address.
IP_NDA_CACHEINFO	Cache statistics.

If the rta_type field is **IP_NDA_CACHEINFO** then a struct nda_cacheinfo header follows.

IP_RTM_NEWRULE, **IP_RTM_DELRULE**, **IP_RTM_GETRULE**

Use this message type to add, delete, or retrieve a routing rule. It carries a struct Ip_rtmsg.

Classes, Types, and Macros

IP_RTA_OK(*rta*, *attrlen*) returns true if rta points to a valid routing attribute. attrlen is the running length of the attribute buffer. When not true, you must assume there are no more attributes in the message, even if *attrlen* is non-zero.

IP_RTA_DATA(*rta*) returns a pointer to the start of this attribute's data.

IP_RTA_PAYLOAD(*rta*) returns the length of this attribute's data.

IP_RTA_NEXT(*rta*, *attrlen*) gets the next attribute after *rta*. Calling this macro update *attrlen*. You should use **IP_RTA_OK** to check the validity of the returned pointer.

IP_RTA_LENGTH(*len*) returns the length which is required for len bytes of data plus the header.

IP_RTA_SPACE(*len*) returns the amount of space needed in a message containing *len* bytes of data.

Previous topic: Macros Reference on page 52

9. PACKET SOCKETS

1. Introduction

Packet sockets are used to send or receive raw packets at the device driver level (OSI Layer 2).

They allow the user to implement protocol modules in user space on top of the physical layer.

Configuring VxWorks for Packet Sockets

By default, any VxWorks image project that is configured with the network stack also contains packet sockets. You do not need to take further action to include packet sockets when you compile the source code or configure a VxWorks image project.

Next topic: Working with Packet Sockets on page 62

2. Working with Packet Sockets

To use a packet socket, specify **AF_PACKET** as the domain in any of the standard socket API functions in the **sockLib** library.

Specify one of the following two values as the socket type:

- **SOCK_RAW** for raw packets including the link level header
- **SOCK_DGRAM** for cooked packets with the link level header removed

The link level header information is available in a common format in an sockaddr_ll structure. For the sll_protocol field, you can specify any of the **IPNET_ETH_P_xxx** constants in network order for the Ethernet value. See the following include file for a list of allowed protocols:

installDir **/vxworks-7/pkgs_v2/net/ipnet/coreip/src/ipnet2/src/ipnet_eth.h**

When a system receives incoming packets of the specified protocol type, it passes them to the packet socket before passing them further to the protocols implemented in the network stack.When protocol is set to htons(IPNET_ETH_P_ALL), the system receives all protocols.

The connect( ) operation is not supported on packet sockets.

Raw Packets

> **SOCK_RAW** packets are passed to and from the device driver without any changes in the packet data. When the system receives a packet, it parses the address and passes it in a standard sockaddr_ll address structure. When a system transmits a packet, the user-supplied buffer must contain the physical layer header. That packet is then queued, unmodified, to the network driver of the interface defined by the destination address.

Cooked Packets

> **SOCK_DGRAM** operates on a slightly higher level. The physical header is removed before the packet is passed to the user. Packets sent through a **SOCK_DGRAM** packet socket get a suitable physical layer header based on the information in the sockaddr_ll destination address before they are queued.

WNDRVR

Socket Options

At the **SOL_PACKET** level, you can use the following options:

- PACKET_ADD_MEMBERSHIP: join a link layer multicast group
- PACKET_DROP_MEMBERSHIP: leave a link layer multicast group

At the **SOL_SOCKET** level, you can use the following options:

- SO_ERROR (get only): read and reset persistent and asynchronous socket error
- SO_SNDBUF: socket send buffer size
- SO_RCVBUF: socket receive buffer size
- SO_TYPE: (get only): read socket type (the second argument passed to socket( )).
- SO_X_VR: set/get the virtual router that owns this socket

No other socket options work for **AF_PACKET**.

Address Types

The structure definition for the sockaddr_ll address type is in

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/ipcom/include/ipcom_sock.h**

The sockaddr_ll is a device-independent physical layer address and contains the following fields:

- sll_protocol: link layer protocol type in network order. The Ethernet link layer protocol is located in **ipnet_eth.h** (**IPNET_ETH_P**_*xxx*). It defaults to the socket's protocol if set to 0.
- sll_ifindex: the interface index of the interface. It must not be 0.
- sll_hatype: an ARP type as defined in the **ipnet_netif.h** include file (**IP_IFT**_*xxx*).
- sll_pkttype: contains the packet type. Valid types are:
  - **PACKET_HOST** for a packet addressed to the local host.
  - **PACKET_BROADCAST** for a physical layer broadcast packet.
  - **PACKET_MULTICAST** for a packet sent to a physical layer multicast address.
  - **PACKET_OTHERHOST** for a packet to some other host that has been caught by a device driver in promiscuous mode.
  - **PACKET_OUTGOING** for a packet originated from the local host that is looped back to a packet socket. These types make only sense for receiving.
- sll_addr and sll_halen: contain the physical layer address and its length. The exact interpretation depends on the device.

sll_ifindex. The other fields should be 0. The sll_hatype and sll_pkttype fields are set for your information on received packets.

The device types (sll_hatype) are defined in the following file:

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/utils/h/net/if_types.h**

Ethernet would be **IFT_ETHER**.

The protocol (or frame) types (sll_protocol) are defined in the following file:

installDir **/vxworks-7/pkgs_v2/net/net_base-***x.x.x.x* **/h/netpacket/packet.h**

Example would be **NET_ETH_P_IP** for IPv4 in Ethernet.

Previous topic: Introduction on page 62

10. ADDING A SOCKET INTERFACE TO A NETWORK PROTOCOL LAYER SERVICE

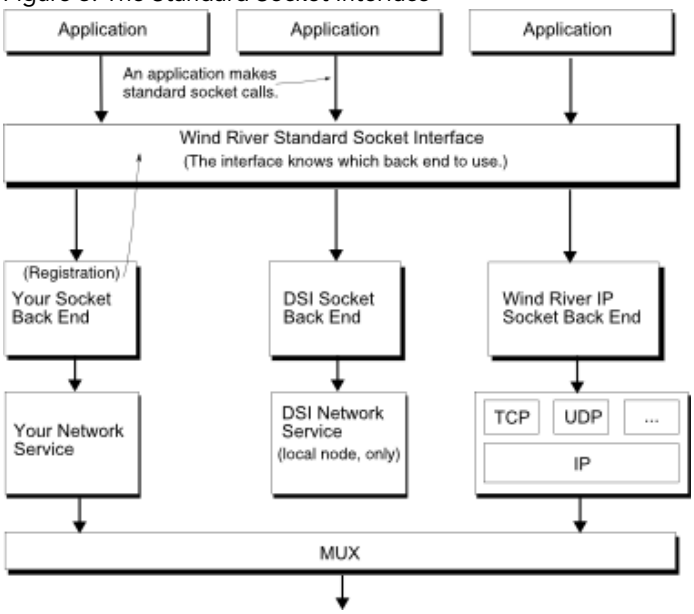
1. The Standard Sockets Interface

One way to allow applications to access your network protocol layer service is to add sockets support to the service.

The Wind River Network Stack includes a standard sockets interface that lets you add new socket back ends to access your network service, and through it, the network.

This allows developers who are already familiar with the standard sockets API to more easily use your service. And, because of the layered architecture— implemented by **sockLib** as a layer above your back end socket layer, as shown in the diagram below— applications can create and use sockets of different address families, which may be managed by different back end service implementations.

Figure 3. The Standard Socket Interface



Next topic: Creating a Socket on page 65

2. Creating a Socket

To create a socket, an application calls the standard function socket( ), and receives a socket descriptor in return.

The socket( ) function looks up the correct back end implementation to use based upon the specified domain (address family) argument. If it finds a registered back end that handles that address family, socket( ) calls the back end's socket creation function, obtaining back a data structure that represents the socket. The socket( ) function then completes the data structure by allocating a descriptor from the I/O system and associating it with the socket data structure, and returns the descriptor to the caller.

The socket data structure contains a pointer to a table of pointers to functions that the back end provides to implement all the various socket operations (see Socket Functional Interface on page 67). You implicitly specify this table when you register the socket back end by calling sockLibAdd( ), as discussed in Registering a Socket Back End on page 66. When an application makes sockets

API calls other than socket( ), the socket descriptor returned by socket( ) is one of the arguments. The **sockLib** implementation for each such sockets API call converts the socket descriptor to the underlying socket data structure using I/O system descriptor look-up facilities, fetches the pointer to the back end's function table, and calls the appropriate back end function to complete the call.

The I/O system functions read( ), write( ), ioctl( ), and close( ) can also be used on socket descriptors. In this case, the I/O system converts the descriptor to the underlying socket data structure, calls the registered **sockLib** read, write, ioctl, or close function, which in turn calls the back end's read, write, ioctl, or close handler via the back end function table. **sockLib** registers a single I/O system driver to handle all socket descriptors.

Note:

The data structure that **sockLib** uses to represent an open socket is the type struct socket. Previous to release SR0650, the struct socket type contained many members inherited from the old pre-IPNET BSD-based VxWorks network stack. Starting in SR0650, the old struct socket containing most of these legacy members is renamed to struct socket_bsd, declared in the header file

installDir **/vxworks-7/pkgs_v2/os/service/socket-**x.x.x.x **/h/net/socketvar.h** The new header file

installDir **/vxworks-7/pkgs_v2/os/service/socket-**x.x.x.x **/h/net/socketcore.h**

which is included in **socketvar.h**, defines a new, much smaller, struct socket structure containing members used by **socklib** as follows:

```
struct socket
{
struct sockFunc * pSockFuncTbl;  /* socket back end function table */
int               so_fd;         /* RTP-specific socket file descriptor */
                                 /* (used for systemviewer events only) */
unsigned int      so_bkend;      /* any back end use / LP64 32-bit align gap */
};
```

The struct socket_bsd starts with the member

```
struct socket     so_so;         /* must be first member */
```

and removes the former **pSockFuncTbl** and **so_fd** members, which are now part of **so_so**.

sockLib bases its interface with its back ends upon the new, smaller, struct socket. Back ends that do not need the legacy members of struct socket_bsd can embed the small struct socket in their own, back end specific socket structure, and ignore struct socket_bsd. If your back end needs members of struct socket_bsd, you can have your back end socket creation function allocate a struct socket_bsd, and cast between that type and the type struct socket of its first member in the back end socket functions. Or, you can copy the needed members from struct socket_bsd into your back end specific socket type.

The interface between **sockLib** and socket back ends, as well as the interface for registering socket back ends, described here, may be modified in future releases.

Previous topic: The Standard Sockets Interface on page 65
Next topic: Registering a Socket Back End on page 66

3. Registering a Socket Back End

You can register a socket back end implementation by calling sockLibAdd( ) some time during system start-up after **sockLib** is itself initialized by a call to sockLibInit( ).

The sockLibAdd( ) function has the following prototype:

```
extern STATUS sockLibAdd (SOCK_LIB_INIT_RTN sockLibInitRtn, int domainMap, int domainReal);
```

The function returns **OK**, or **ERROR** if the function could not add the socket back end.

The function takes three parameters:

sockLibInitRtn

A pointer to your xSockLibInit( ) function that sockLibAdd( ) invokes. sockLibAdd( ) calls this function as if it had the following prototype:

```
SOCK_FUNC * xSockLibInit (void);
```

That is, it is passed no arguments, and it is expected to return a pointer to an initialized **SOCK_FUNC** structure, which is the table of function pointers for the back end (see Socket Functional Interface on page 67).

Note: **sockLib** will not modify the returned **SOCK_FUNC** structure, so it may be statically allocated.

domainMap

sockLibAdd( ) ignores this parameter.

domainReal

This parameter specifies the address family that this back end implements. A back end may support more than one address family, but in such a case you must call sockLibAdd( ) multiple times, once per address family. Allowed address families are in the range from 1 to **AF_MAX-1**, and the **AF_** constants that identify these address families (**AF_INET**, and so forth) are declared in

installDir **/vxworks-7/pkgs_v2/os/service/socket-***x.x.x.x* **/h/sys/socket.h**

At most, one back end will handle sockets of any given address family. For example, the native Wind River Network Stack normally handles sockets of the **AF_INET**, **AF_INET6**, **AF_ROUTE**, and **AF_PACKET** families. A socket back end of your own implementation may not handle one of these address families without disabling handling of that family by the native stack.

Previous topic: Creating a Socket on page 65
Next topic: Socket Functional Interface on page 67

4. Socket Functional Interface

The socket functional interface is the set of implementations of standard socket functions that a particular socket back end supports.

SOCK_FUNC is declared in

installDir **/vxworks-7/pkgs_v2/os/service/socket-***x.x.x.x* **/h/sockFunc.h**

as follows:

```
struct sockFunc                 /* SOCK_FUNC */
        {
        SOCK_FUNC_INIT              libInitRtn; /* Unused */
        SOCK_FUNC_ACCEPT            acceptRtn;
        SOCK_FUNC_BIND              bindRtn;
        SOCK_FUNC_CONNECT           connectRtn;
        SOCK_FUNC_CONN_TIMEO        connectWithTimeoutRtn;
```

```
        SOCK_FUNC_GETPEERNAME       getpeernameRtn;
        SOCK_FUNC_GETSOCKNAME       getsocknameRtn;
        SOCK_FUNC_LISTEN            listenRtn;
        SOCK_FUNC_RECV              recvRtn;
        SOCK_FUNC_RECVFROM          recvfromRtn;
        SOCK_FUNC_RECVMSG           recvmsgRtn;
        SOCK_FUNC_SEND              sendRtn;
        SOCK_FUNC_SENDTO            sendtoRtn;
        SOCK_FUNC_SENDMSG           sendmsgRtn;
        SOCK_FUNC_SHUTDOWN          shutdownRtn;
        SOCK_FUNC_SOCKET            socketRtn;
        SOCK_FUNC_GETSOCKOPT        getsockoptRtn;
        SOCK_FUNC_SETSOCKOPT        setsockoptRtn;
        SOCK_FUNC_ZBUF              zbufRtn;

        /* The following IO-system handlers are called via wrappers in socklib.c */

        SOCK_FUNC_CLOSE             closeRtn;
        SOCK_FUNC_READ              readRtn;
        SOCK_FUNC_WRITE             writeRtn;
        SOCK_FUNC_IOCTL             ioctlRtn;
        };
```

The sockFunc header also has typedefs for each of the function types used in this structure, such as **SOCK_FUNC_ACCEPT**:

```
typedef int (*SOCK_FUNC_ACCEPT)
    (struct socket **pso,
    struct sockaddr * addr,
    socklen_t * pAddrlen);
```

In most cases, you can guess the type of each **SOCK_FUNC** function by considering the corresponding standard sockets API function prototype in **sockLib.h**, or the corresponding I/O system function prototype in **ioLib.h**, and replacing the integer socket descriptor argument with a struct socket * argument. For instance, the connect API is prototyped in **sockLib.h** as the following:

```
extern STATUS connect (int s, const struct sockaddr *name, socklen_t namelen);
```

The back end function called through the connectRtn( ) function pointer is called as if it had the following prototype:

```
STATUS xConnectRtn (struct socket * so, const struct sockaddr * name, socklen_t namelen);
```

The **sockLib** connect( ) implementation converts the integer socket descriptor passed as its first argument, s, to the struct socket pointer so, and calls the back end's *x*ConnectRtn( ) function, replacing s with so and passing the name and namelen arguments unchanged. From the so argument, the socket back end can find its own private data for the socket. Typically the small struct socket is embedded in the back end's private socket data structure, at a known offset. Alternatively, the back end can use the so_bkend member of the struct socket pointed to by so, as an index or key to locate the back end private socket data. The so_bkend member is reserved for any back end use.

The return value from the back end *x*ConnectRtn( ) is same as the return value from connect.

Consult the **sockLib** reference pages for additional information on the intended behavior of the sockets API functions. External sockets API information is also very useful; for instance IEEE Std 1003.1 contains the official POSIX descriptions of sockets API functions and data structures. (Historically, the VxWorks sockets API prototypes did not always match exactly those defined by POSIX; but in VxWorks 6.9 and later versions, the socket API prototypes are consistent with the POSIX sockets prototypes.)

There are exceptions to the above rule-of-thumb. These are socketRtn( ), acceptRtn( ), and ioctlRtn( ).

*x*SocketRtn( )

The *x*SocketRtn( ) function has the following prototype:

```
STATUS xSocketRtn
    (
    int              domain,     /* socket domain or address family number */
    int              type,       /* socket's nature, e.g. SOCK_DGRAM */
    int              protocol,   /* the protocol variety of the socket */
    struct socket ** ppSo        /* the socket structure */
    )
```

The back end's *x*SocketRtn( ) is responsible for allocating and initializing a struct socket and any other structures that the back end needs to represent a socket of the specified domain, type, and protocol (these are passed directly from the corresponding arguments to the socket( ) function). The allocation may be done using the kernel heap, a **netBufLib** pool, or a back-end specific method. If the *x*SocketRtn( ) cannot allocate and initialize a socket of the specified kind, xSocketRtn( ) must free any partial allocations, set errno to the appropriate value (such as **ENOMEM**, **ENOBUFS**, **EAFNOSUPPORT**, **EPROTONOSUPPORT**, **EPROTOTYPE**), and return **ERROR**.

Otherwise, *x*SocketRtn( ) stores a pointer to the allocated struct socket at the address specified by ppSo, and returns **OK**. The socket( ) code will then itself store a pointer to the back end's **SOCK_FUNC** table in the returned struct socket's pSockFuncTbl member, then attempt to allocate a file descriptor from the I/O system, associating it with the socket. The socket( ) call may still fail if the calling RTP (possibly the kernel) is out of file descriptors. In this case, the socket( ) code will immediately call the back end's *x*CloseRtn( ) function, to destroy the socket, and return **ERROR**. On the other hand, if a file descriptor is successfully allocated, the socket( ) code stores that file descriptor in the so_fd member, and return the file descriptor as its result.

sockLib itself has no requirements for how the back end initializes the struct socket structure that it allocates. **sockLib** initializes the pSockFuncTbl and so_fd members itself, and ignores the so_bkend member, which the back end can set however it wants.

*x*AcceptRtn( )

The *x*AcceptRtn( ) function has the following prototype:

```
STATUS xAcceptRtn
        (
        struct socket ** ppSo,  /* IN: parent socket. OUT: child socket. */
        struct sockaddr * addr, /* Address of child's peer. */
        socklen_t * addrlen     /* Length of child's peer's address. */
        );
```

The back end's *x*AcceptRtn( ) is called by the accept( ) code in **sockLib**. This accept( ) code does some checking, however, before it calls *x*AcceptRtn( ). If addr is non-NULL but addrlen is NULL, accept( ) returns an error. Otherwise, accept( ) attempts to convert the descriptor passed as its first argument to a struct socket. If the descriptor is not a valid socket descriptor, accept( ) again returns an error. If the back end does not provide any accept handler, that is, if the acceptRtn member of the back end's **SOCK_FUNC** structure is NULL, accept( ) again returns **ERROR**.

Otherwise, the accept( ) code calls the back end's *x*AcceptRtn( ) function, passing as the first argument the address of a pointer to the struct socket, converted from the socket descriptor passed to accept( ). The other two arguments to *x*AcceptRtn( ) are passed directly from the corresponding arguments of accept( ).

The back end's *x*AcceptRtn( ) must check that the socket passed in by the first argument is in fact a listening parent socket, capable of providing child socket connections to accept( ). If not, *x*AcceptRtn( ) must return **ERROR** and set errno appropriately (see IEEE Std 1003.1). If the parent can provide child connections, but no completed child socket connection is presently queued for the parent, then *x*AcceptRtn( ) must do one of the following:

- Set errno to **EAGAIN** or **EWOULDBLOCK** and return **ERROR**, if the parent socket is non-blocking.
- If the parent socket is blocking, pend until either a completed child socket connection becomes available (or optionally: until a timeout, signal, or other implementation defined event occurs, in which case errno should be set appropriately, and **ERROR** returned).

If a completed child socket connection becomes available, *x*AcceptRtn( ) allocates a struct socket for it (and any other needed private data structures), and stores a pointer to the child struct socket at the address passed in the ppSo argument, overwriting the previous pointer to the parent's struct socket. If the addr argument is non-NULL, *x*AcceptRtn( ) obtains the child's peer's socket address, and copies it (truncating to the length specified in the int pointed to by addrlen, if necessary) to the specified address addr; and finally storing the actual address length in the integer pointed to by addrlen. *x*AcceptRtn( ) then returns OK.

If *x*AcceptRtn( ) does not return **ERROR**, the accept( ) code stores a pointer to the back end's **SOCK_FUNC** table in the child socket's pSockFuncTbl member, then goes on to attempt to allocate a socket descriptor from the I/O system for the child socket. If this fails, accept( ) immediately calls the back end's *x*CloseRtn( ) function and returns **ERROR**.

Otherwise, accept( ) stores the allocated socket descriptor in the child struct socket's so_fd member, and returns that socket descriptor as its result.

*x*IoctlRtn( )

The *x*IoctlRtn( ) function has the following prototype:

```
int xIoctlRtn
        (
        struct socket * so, /* the socket */
        int code,           /* ioctl command code */
        _Vx_usr_arg_t data, /* ioctl argument */
    void * mode         /* indicates if the call is from user space */
        );
```

The back end's *x*IoctlRtn( ) function is called when the I/O system processes an ioctl( ) call made on a socket descriptor. The function is called as the rule of thumb would suggest, passing the struct socket pointer so instead of a socket file descriptor and passing the ioctl command and data arguments unchanged, except that *x*IoctlRtn( ) is also passed another argument mode. This argument is NULL if ioctl( ) was called from the kernel; it is an arbitrary non-NULL value if ioctl( ) was called from a user-space RTP. This indication is intended to help support validation of user-space ioctl arguments.

Previous topic: Registering a Socket Back End on page 66
Next topic: Memory Validation and Socket Ioctls on page 70

5. Memory Validation and Socket Ioctls

Socket API functions are expected to validate their arguments for proper memory access when called from a user-space RTP application.

This is accomplished for most socket calls in the **socketScLib** shim library. This contains the system call handlers for sockets API system calls; these handlers execute in the kernel and perform argument memory access validation before calling the appropriate kernel socket API functions in **sockLib**. (Memory validation is done using the scMemValidate( ) function; see its reference entry for more information.)

Memory validation for the read( ) and write( ) buffer and length arguments is done by the I/O system's system call handler code. However, ioctl( ) calls made on socket descriptors are a special case. The I/O system level does not have knowledge about the form and use of the ioctl arguments passed to ioctl operations implemented by the lower-level "driver" code, such as the socket back ends, and so cannot do the memory validation. In VxWorks, the lower-level drivers (including socket back ends) are expected to do memory validation for the ioctls they implement which may be called by RTP applications.

sockScLib provides a pair of functions that can aid a socket back end in doing ioctl argument memory validation. These functions should only be called through the following function pointers:

```
extern int (*pSockIoctlMemVal)
(
   int cmd,
```

```
    const void * data
  );
extern STATUS (*pUnixIoctlMemVal)
(
    int cmd,
    const void * pData
  );
```

The function pointers are only non-NULL when RTP support is included in the VxWorks image. They should only be called when the mode argument passed to xIoctlRtn( ) is non-NULL, indicating an ioctl( ) call from user space.

Most (although unfortunately not all) ioctl commands on sockets encode the length and usage of the command argument in the ioctl command code itself. Ioctl codes following the conventions in

installDir **/vxworks-7/pkgs_v2/os/service/socket-***x.x.x.x* **/h/net/ioctl.h**

or

installDir **/vxworks-7/pkgs_v2/net/ipnet-***x.x.x.x* **/coreip-***x.x.x.x* **/src/utils/shareh/ipnet/ipioctl.h**

encode whether the ioctl argument is a pointer to a buffer that is read into the kernel, written to by the kernel, or both; and if so, how large the buffer is. The top-level memory validation for such ioctl codes may be performed by calling pUnixIoctlMemVal( ), passing it the command code and the ioctl data argument. This function returns **OK** if memory validation succeeds, otherwise it sets errno appropriately and returns **ERROR**.

There are some limitations of the function referenced by pUnixIoctlMemVal( ):

- It does not check that the ioctl code is supported by the back end.
- It assumes without any check that the code follows the conventions encoding the parameter length and usage, as described in **ioctl.h**.
- It only does top-level checking: if the ioctl parameter is a pointer to a buffer holding a data structure that contains additional pointers, these other pointers are not validated. Validating them is the responsibility of the back end.

The pSockIoctlMemVal( ) function pointer behaves similarly to pUnixIoctlMemVal( ), except that it additionally validates memory for a small number of ioctl( ) codes that do not follow the conventions upon which pUnixIoctlMemVal( ) depends. In the present release, these additional ioctl codes are **FIONBIO**, **FIONREAD**, **FIOSELECT**, and **FIOUNSELECT**. It also supports **SIOCMUXPASSTHRU** and **SIOCMUXL2PASSTHRU**, doing second-level validation of the embedded MUX ioctl commands in these two ioctls' arguments. For any other ioctl code passed to pSockIoctlMemVal( ), it simply calls pUnixIoctlMemVal( ).

Previous topic: Socket Functional Interface on page 67

WNDRVR

11. NETWORK SOCKET EVENT POLLING

1. Introduction

The VxWorks event polling facility (epoll) is a scalable I/O event notification mechanism for monitoring networking socket events.

A typical application for epoll is in a web server environment that currently uses select( ) to monitor network socket events. The epoll mechanism provides better performance, which is important when monitoring very large number of file descriptors in demanding applications.

Superior performance over select( ) is accounted for by the following:

- The epoll facility de-couples monitor registration from the actual monitoring of the events.
- The select( ) requires the full list of file descriptors to watch each invocation.
- Using epoll with the EPOLLONESHOT option, a file descriptor can be removed automatically from the monitoring set after an event is generated and read, without explicitly calling an additional epoll API to remove the file descriptor.

The epoll facility provides better scalability when compared to select( ) because:

- The maximum number of file descriptors that can be monitored by select( ) are limited by fd_set (with a maximum of 2048), while epoll has no such limitation.
- The epoll facility can be used with both uni-core (UP) and multi-core (SMP) VxWorks systems.

VxWorks epoll also supports both level-triggered and edge-triggered event monitoring modes, whereas select( ) only monitors level-triggered events.

The VxWorks epoll API functions have the same syntax as the comparable Linux epoll API functions.

The source code files for epoll are in the following directory:

installDir **/vxworks-7/pkgs_v2/os/service/epoll-***x.x.x.x* **/**

Next topic: Source Build Configuration Options (VSB) on page 72

2. Source Build Configuration Options (VSB)

Three options are available for the EPOLL facility.

EPOLL

Enable the VxWorks event polling (epoll) facility, a scalable input and output event notification mechanism.

The SOCKET and IPNET_COREIP options must also be enabled.

POLL_ALL_ENTRIES

The POLL_ALL_ENTRIES option is enabled by default.

When enabled, epoll checks all entries in the epoll instance, but only polls the file descriptors that have events or need to be polled for level-triggered monitoring at epoll_wait( ). This option is defined by default to get better performance when a lot of monitored file descriptors have concurrent events.

WNDRVR

It should only be disabled in cases when there are of a lot of monitored file descriptors, but few of them have concurrent events.

EPOLL_IPNET_SOCK_USE_SIGNALS

If a socket will be polled by multiple instances of epoll, enable EPOLL_IPNET_SOCK_USE_SIGNALS (which is disabled by default).

When this option is enabled, epoll registers and de-registers the event watch through an **IPNET** pipe signal without the mutex semaphore.

When disabled, epoll directly registers and de-registers the event watch in a socket—adding/removing a monitored socket to/from the epoll list directly, with protection by a mutex semaphore.

Previous topic: Introduction on page 72
Next topic: Kernel Configuration Components (VIP) Reference on page 73

3. Kernel Configuration Components (VIP) Reference

The INCLUDE_EPOLL component is used to configure the kernel configuration.

Previous topic: Source Build Configuration Options (VSB) on page 72
Next topic: Event Polling API on page 73

4. Event Polling API

The following table lists the API functions available to epoll applications and gives a brief description of each function. For more information, see the reference entries for epoll (**epollLib**). The API functions are defined in **epollLib.c**.

Table 1. epoll API Functions

Function	Description
epoll_create( )	Creates a new epoll instance/context. This function has been superseded by **epoll_create1( )**.
epoll_create1( )	Creates a new epoll instance.
epoll_ctl( )	Adds, removes, or modifies monitored file descriptors on the epoll instance.
epoll_wait( )	Waits for events on a set of file descriptors.
epoll_create_and_ctl( )	Creates a new epoll instance, and configures the set of I/O events of interest for a file descriptor.
close( )	Close and free a given epoll file descriptor and epoll instance.

The epoll_create_and_ctl( ) function is a VxWorks extension. It is provided for better usage and performance allowing the user to make one system call—instead of making two separate calls with epoll_create( ) and epoll_ctl( )—to create and initialize the epoll instance.

To add, remove, or modify the monitored file descriptors that are managed with an epoll instance, use EPOLL_CTL_ADD, EPOLL_CTL_DEL, or EPOLL_CTL_MOD (respectively) as the second parameter for epoll_ctl( ). The epoll_ctl( ) function configures a set of I/O events of interest for a file descriptor with EPOLL_CTL_MOD.

To define the epoll events that should be monitored, use one or more of the following to populate the events element of structure that is identified with the third parameter for epoll_ctl( ):

Option	Description
EPOLLIN	Socket is readable
EPOLLPRI	Urgent data present
EPOLLOUT	Socket is writable
EPOLLERR	Pending error condition
EPOLLHUP	Hang-up occurred, unused by IPNET
EPOLLRDHUP	Peer closed its write end
EPOLLONESHOT	Set one-shot behavior
EPOLLET	Edge triggered poll

When the EPOLLET option is used, the edge-triggered mode is enabled. In edge-triggered mode epoll_wait( ) will return only on the occurrence of an expected new event or after a timeout. Once the new event is registered, subsequent calls to epoll_wait( ) in edge-triggered mode will not return on the same event that is not cleared. (The event may not have been cleared due to pending input data).

When EPOLLET option is not used, level-triggered mode is enabled. In level-triggered mode every subsequent call to epoll_wait( ) will successfully return until the event is cleared.

Previous topic: Kernel Configuration Components (VIP) Reference on page 73
Next topic: Example epoll_wait( ) Usage in an Edge-Triggered Environment on page 74

5. Example epoll_wait( ) Usage in an Edge-Triggered Environment

Example 1:

1. Some events happen on the file descriptor (fd).
2. The fd is added to the epoll-set in edge-triggered mode.
3. epoll_wait( ) is called and the fd will be reported as ready.
4. epoll_wait( ) is called and the fd will not be reported as ready.

Example 2:

1. The fd is added to the epoll-set in edge-triggered mode.
2. Some events happen on the fd; for example, 2 KB of data arrives.
3. epoll_wait( ) is called and the fd will be reported as ready.
4. Read 1 KB of data from the fd.
5. epoll_wait( ) is called again, the fd will not be reported as ready in edge-triggered mode. This behavior is different from the level-triggered mode where the fd will be reported as ready.

Example 3:

1. The fd is added to the epoll-set in edge-triggered mode.
2. epoll_wait( ) is called and the fd will not be reported as ready.
3. Some events happen on the fd.
4. Some more events happen on the fd.
5. epoll_wait( ) is called and the fd will be reported as ready.
6. epoll_wait( ) is called again, the fd will not be reported as ready.

Example 4:

1. The fd is added to the epoll-set in edge-triggered mode.
2. Some events happen on the fd; for example, 2 KB of data arrives.
3. epoll_wait( ) is called and the fd will be reported as read ready
4. Read 1 KB of data from the fd.
5. Some events occur on the fd; for example, write space opens up, TCP peer acknowledges data, and the fd state is read ready and write ready.
6. epoll_wait( ) is called and the fd will be reported as read ready and write ready.

Previous topic: Event Polling API on page 73
Next topic: Using Event Polling in an Application on page 75

6. Using Event Polling in an Application

Using event polling in an application generally involves four tasks.

1. Include the epoll header file **epoll.h**.
2. Create and initialize an event polling instance. The epoll_create_and_ctl( ) function can be used instead of two separate calls for creation and initialization.
3. Perform the event wait with epoll_wait( ). Note that when a file descriptor (fd) is added or modified in the epoll instance, and the events on the fd have already occurred, the next call to epoll_wait( ) will report the events on the fd.
4. Add, remove, or modify monitored file descriptors on the epoll instance as needed with epoll_ctl( ).

The following code shows how to use event polling functions:

```
#define MAX_EVENTS 10
struct epoll_event ev, events[MAX_EVENTS];
int listen_sock, conn_sock, nfds, epollfd;

/* Set up listening socket 'listen_sock' */

ev.events = EPOLLIN;
ev.data.fd = listen_sock;

epollfd = epoll_create_and_ctl(1, &listen_sock, &ev);
if (epollfd == -1) {
        perror("epoll_create_and_ctl");
        exit(EXIT_FAILURE);
}

for (;;) {
        nfds = epoll_wait(epollfd, events, MAX_EVENTS, -1);
        if (nfds == -1) {
                perror("epoll_wait");
                exit(EXIT_FAILURE);
        }

        for (n = 0; n < nfds; ++n) {
                if (events[n].data.fd == listen_sock) {
                        conn_sock = accept(listen_sock,
                                                (struct sockaddr *)&local, &addrlen);
                        if (conn_sock == -1) {
                                perror("accept");
                                exit(EXIT_FAILURE);
                        }

                        ev.events = EPOLLIN;
                        ev.data.fd = conn_sock;
                        if (epoll_ctl(epollfd, EPOLL_CTL_ADD, conn_sock,
                                                &ev) == -1) {
                                perror("epoll_ctl: conn_sock");
                                exit(EXIT_FAILURE);
                        }
                } else {
                        /* Process data arrived on a data socket */
                }
        }
}
```

Previous topic: Example epoll_wait( ) Usage in an Edge-Triggered Environment on page 74
Next topic: Limitations of Event Polling on page 76

7. Limitations of Event Polling

You should understand the limitations of event polling before implementing a solution.

- Currently only the socket file descriptors can be added to the epoll monitored file descriptors.
- An epoll instance cannot be added to the monitored file descriptors of another epoll instance.
- For a single epoll instance, more than one task cannot be calling epoll API functions concurrently.

- Epoll is only supported in the kernel space. Support for epoll in the RTPs is not available. Even if you have added support for RTPs in VxWorks, file descriptors from multiple RTPs cannot be added to the same epoll instance. That is, an epoll instance can only be used by one RTP. However multiple RTPs and their corresponding epoll instances can monitor the same socket.

Previous topic: Using Event Polling in an Application on page 75